

SCHOOL OF INFORMATION TECHNOLOGY AND ENGINEERING

BCSE308P COURSE PROJECT

COMPUTER NETWORKS

PRESTO-R: Predictive, Stability-Aware Reinforcement Learning Routing

Formal Project Final Report

Submitted by	Yashwanth Gajula
Registration Number	24BDE0079
Slot	C2+TC2
Faculty Guide	Dr. Navaneethan C
Semester	Fall Semester 2026-2027
Date	23 October 2026

VIT Vellore

DECLARATION

I, Yashwanth Gajula, Registration Number 24BDE0079, declare that this report presents the PRESTO-R Computer Networks project and identifies its methods, data sources, software, evaluation procedures, results, limitations, and references. The numerical results are transcribed from the saved project outputs and are presented with their stated aggregation and uncertainty conventions.

I have distinguished project-generated evidence from interpretation and have cited the research and software sources used to describe the methods. The reported evaluation is limited to the simulator configurations, scenarios, seeds, and saved model versions documented in the project repository.

Place: Vellore, Tamil Nadu

Date: 23 October 2026

Signature of the Candidate: _____

CERTIFICATE

This report documents the project titled PRESTO-R: Predictive, Stability-Aware Reinforcement Learning Routing, submitted by Yashwanth Gajula (24BDE0079) for BCSE308P Computer Networks at VIT Vellore under the guidance of Dr. Navaneethan C.

The report records the project design, implementation, experimental evaluation, findings, and limitations. Faculty review and certification are reserved for the authorized guide.

Faculty Guide: Dr. Navaneethan C

Signature: _____

Date: _____

Place: Vellore

ACKNOWLEDGEMENT

I express my sincere gratitude to Dr. Navaneethan C for academic guidance and feedback during the Computer Networks project. I also acknowledge VIT Vellore for the academic setting in which the work was developed.

I am grateful to the open-source maintainers whose libraries support the simulator, machine-learning workflow, statistical analysis, and visualizations. The report preserves the project's recorded results, including unsuccessful criteria and methodological limitations, so that the conclusions remain traceable to the evidence.

Yashwanth Gajula

24BDE0079

EXECUTIVE SUMMARY

PRESTO-R combines link-degradation prediction with deep reinforcement learning and a penalty for changing routes. The project implements a discrete-time fluid-network simulator, episode-separated telemetry datasets, logistic-regression and GRU predictors, reactive and predictive DQN baselines, and a stability-aware DQN routing policy. The completed evaluation covers NSFNET and random geometric graph topologies, five traffic and failure scenarios, held-out scenario seeds, horizon and penalty sweeps, and three robustness conditions. In NSFNET mixed episodes, PRESTO-R averaged 5.197 route switches and PDR 0.9625, compared with 20.988 switches and PDR 0.9608 for predictive DQN. In RGG mixed episodes, PRESTO-R averaged 1.913 switches and PDR 0.9545, compared with 11.873 switches and PDR 0.9546. The largest unresolved gap is reliable early warning: GRU onset F1 was 0.5296 against a target of 0.7000. PRESTO-R did not pass every comparison with reactive DQN, and the no-prediction plus selected-penalty factorial cell was not run. Findings therefore support reduced route churn in the tested simulator, not production-network performance.

Word count: 155

CONTENTS

CHAPTER 1 INTRODUCTION 1
1.1 Objective 1
1.2 Motivation 2
1.3 Background 3
1.4 Problem Statement 4
1.5 Gap Identification 4
1.6 Objectives and Evaluation Alignment 5
1.7 Scope and Contributions 6
CHAPTER 2 PROJECT DESCRIPTION AND GOALS 7
2.1 Project Overview 7
2.2 Goals and Acceptance Criteria 8
2.3 System Boundary 9
2.4 Actors and Operating Workflow 9
2.5 Assumptions 10
2.6 Evidence and Deliverables 10
CHAPTER 3 TECHNICAL SPECIFICATION 12
3.1 Network and Simulator Specification 12
3.2 Topology Specification 12
3.3 Traffic and Failure Specification 13
3.4 Telemetry and State Specification 14
3.5 Predictor and Agent Specification 15
3.6 Software Stack 15
3.7 Data and Reproducibility Specification 16
3.8 Metric Definitions 17
CHAPTER 4 DESIGN APPROACH AND DETAILS 19
4.1 Design Approach Materials and Methods 19
4.2 Codes and Standards 24
4.3 Constraints Alternatives and Tradeoffs 25
4.4 Statistical Design and Validity 26
CHAPTER 5 SCHEDULE TASKS AND MILESTONES 28
5.1 Work Breakdown 28
5.2 Phase Schedule and Status 29
5.3 Milestone Evidence 29
5.4 Reproducibility and Release 30
CHAPTER 6 PROJECT DEMONSTRATION 31
6.1 Sample Codes 31
6.2 Sample Screenshots 35
6.3 End-to-End Demonstration 36
CHAPTER 7 COST ANALYSIS RESULT AND DISCUSSION 38

7.1 Cost and Resource Analysis 38
7.2 Evaluation Protocol 39
7.3 Predictor Results 39
7.4 NSFNET Results 45
7.5 Paired Tests 50
7.6 Ablation and Penalty Sweep 53
7.7 Horizon Sensitivity 55
7.8 RGG Results 56
7.9 Robustness Evaluation 60
7.10 Success Criteria 63
7.11 Findings by Scenario 65
7.12 Threats to Validity 65
CHAPTER 8 SUMMARY 67
8.1 Objective Achievement 67
8.2 Conclusions 67
8.3 Recommendations 68
8.4 Future Work 68
CHAPTER 9 REFERENCES 70
APPENDIX A REPRODUCTION AND DATA DICTIONARY 72

LIST OF FIGURES

Fig. 4.1 PRESTO-R System Architecture 20
Fig. 4.2 NSFNET Topology Used In The Main Evaluation 20
Fig. 6.1 Representative Training Curves 35
Fig. 6.2 Held-Out Event Trace Demonstration 36
Fig. 7.1 Held-Out Predictor Quality 40
Fig. 7.2 Learned And Oracle Link-Risk Trace 41
Fig. 7.3 NSFNET Main Results 46
Fig. 7.4 Mixed-Scenario Switch-Penalty Trade-Off 54
Fig. 7.5 Prediction-Horizon Sensitivity 56
Fig. 7.6 RGG Main Results 57

LIST OF TABLES

Table 1.1 Evaluation Criteria And Evidence Map 6
Table 2.1 Project Goals And Measures 8
Table 3.1 Core Simulation Parameters 13
Table 3.2 Scenario Configuration 14
Table 3.3 Software Dependencies 16
Table 4.1 Design Alternatives And Trade-Offs 25
Table 5.1 Phase Schedule And Completion Status 28
Table 7.1 Observed Computational Assets And Unrecorded Cost Items 38
Table 7.2 GRU Test Classification And Sample Counts 41
Table 7.3 GRU Test Ranking Calibration And Lead Time 41
Table 7.4 LOGREG Test Classification And Sample Counts 42
Table 7.5 LOGREG Test Ranking Calibration And Lead Time 42
Table 7.6 GRU H=3 Classification And Sample Counts 43
Table 7.7 GRU H=3 Ranking Calibration And Lead Time 44
Table 7.8 GRU H=10 Classification And Sample Counts 44
Table 7.9 GRU H=10 Ranking Calibration And Lead Time 45
Table 7.10 NSFNET Delivery Delay And Recovery (Mean +/- Standard Deviation Across Training Seeds) 46
Table 7.11 NSFNET Switching Proactive Response And Unnecessary Switches (Mean +/- Standard Deviation Across Training Seeds) 47
Table 7.12 Paired Wilcoxon Results With Holm Correction 51
Table 7.13 Paired-Test Signed Effect Sizes 52
Table 7.14 Available Ablation Cells 54
Table 7.15 Lambda Sweep PDR And Switches 54
Table 7.16 Lambda Sweep Unnecessary Switching And Recovery 55
Table 7.17 Lambda Sweep Proactive Rate 55
Table 7.18 Horizon Sweep PDR And Switching 56
Table 7.19 Horizon Sweep Proactivity Recovery And Predictor F1 56
Table 7.20 RGG Delivery Delay And Recovery 57
Table 7.21 RGG Switching Proactivity And Unnecessary Switching 58
Table 7.22 Evaluation-Only Robustness Results 61
Table 7.23 Predeclared Success Criteria 64
Table 7.24 Success-Criteria And Summary Aggregation Difference 64

LIST OF ABBREVIATIONS

Table 0.1 Abbreviations Used In This Report

Abbreviation	Meaning
AI	Artificial Intelligence
AUC	Area Under The Curve
CPU	Central Processing Unit
DQN	Deep Q-Network
F1	Harmonic Mean Of Precision And Recall
GRU	Gated Recurrent Unit
H	Prediction Horizon
NSFNET	National Science Foundation Network Topology
PDR	Packet Delivery Ratio
PR-AUC	Area Under The Precision-Recall Curve
RGG	Random Geometric Graph
RL	Reinforcement Learning
SB3	Stable-Baselines3
SDE	Standard Deviation Across Training-Seed Means
TNR	Times New Roman
VIT	Vellore Institute Of Technology
W	Telemetry Window Length

SYMBOLS AND NOTATIONS

Table 0.2 Symbols And Notations

Symbol	Meaning
α	Delay weight in the reward
β	Loss weight in the reward
γ	Throughput weight in the reward
λ	Route-switch penalty
μ	Hop or energy weight
H	Prediction and event-analysis horizon
W	Telemetry history window
PDR	Delivered traffic divided by offered demand
p_Holm	Holm-adjusted paired-test p-value
r	Signed effect-size statistic reported by paired tests
σ	Across-training-seed standard deviation

CHAPTER 1
INTRODUCTION

Computer networks carry traffic over paths whose quality changes as load, link health, and failures evolve. A routing policy must preserve delivery and recover from faults without destabilizing the network through excessive route changes. PRESTO-R examines a specific design hypothesis: adding predicted link risk to a learned route selector can help it anticipate degradation, while a switching cost can restrain unnecessary path changes. The project implements that hypothesis in a controlled simulator and evaluates it against reactive learning, prediction without the selected penalty, and shortest-path failover. The central result is a substantial reduction in route switching in the evaluated mixed scenarios, with PDR remaining close to the learned baselines. The evidence also identifies an unresolved weakness: the predictor misses the predeclared onset-F1 target, and PRESTO-R fails some comparisons against reactive DQN.

1.1 Objective

The primary objective is to design, implement, and evaluate a route-selection system that augments current path measurements with an estimate of near-future link degradation. The action space consists of four candidate paths selected for a managed source-destination flow. The system must expose a stable observation vector, compute an interpretable reward, and record enough event-level truth to evaluate delivery, switching, recovery, proactive response, and prediction quality.

A second objective is to test whether a route-switch penalty reduces route churn without materially reducing packet delivery. The policy comparison separates three effects: reactive DQN observes current measurements only; predictive DQN adds learned risk; PRESTO-R adds risk and the selected switching penalty. A shortest-path failover heuristic supplies a non-learning reference. These internal baselines do not claim to reproduce external routing algorithms beyond their implemented behavior.

A third objective is to characterize the predictor itself. Logistic regression and a GRU are trained from simulator-derived telemetry, and performance is reported on all samples and on the onset subset. Subgroup results distinguish scenario, topology, and

event type. This distinction matters because an all-sample score can look strong when healthy observations dominate, while onset detection remains weak.

The evaluation objective is to compare these systems under the same held-out scenario seeds across NSFNET and a random geometric graph. It includes steady, congestion, node-failure, fluctuation, and mixed scenarios, plus horizon sensitivity and evaluation-only robustness conditions. Test data is reserved for final measurement and is not used to retune the policy or risk threshold.

1.2 Motivation

Reactive routing observes a path after its delay, loss, or availability has already changed. Such feedback can be sufficient for many systems, but a policy that selects among candidate paths at each step may react to noisy or transient measurements. Repeated switching is costly even when a new path is not meaningfully better: it can complicate operations, produce unstable forwarding decisions, and create transient performance penalties. The project therefore treats route stability as a measurable outcome alongside packet delivery.

Prediction provides a possible earlier signal. A temporal model can use recent link telemetry to estimate whether a link will become degraded within a selected horizon. A routing policy can then include risk as one of its path features. The signal is imperfect: the predictor can produce false alarms, miss sudden failures, or generalize poorly across scenarios. For this reason, PRESTO-R is evaluated on held-out episodes and compared with a policy that has the same predictive features but no selected switch cost.

A switching penalty provides a second control. It discourages changing routes for small or temporary differences, but it can also delay necessary movement. The design question is not whether switching is always harmful; switching is useful when a selected route is failing. The objective is to reduce avoidable switches while preserving delivery and recovery. The project reports proactive rate and recovery time to make this balance visible rather than relying on switch count alone.

The project uses simulation because the full training grid requires repeatable interactions across many combinations of topology, event type, and seed. A discrete-time fluid model permits many rollouts at manageable cost. The controlled setting supports

reproducible comparisons, but it also limits external validity. It cannot reproduce every packet-level, transport, routing-protocol, or operational effect of a production network.

1.3 Background

Reinforcement learning models sequential decisions as interactions between an agent and an environment. At each time step, the agent receives an observation, chooses an action, and receives a reward. A DQN estimates the value of each available action from the observation and selects a high-value action. Mnih and colleagues demonstrated deep Q-networks as a method for learning action values with neural representations [1]. PRESTO-R uses the same broad DQN family through the project's Stable-Baselines3 training interface; its network states and action meanings are specific to this routing simulator.

A GRU is a recurrent neural network unit intended to summarize sequence history. In this project, the predictor reads a window of recent telemetry for each link and emits a probability that the link will be bad within a future horizon. The use of a sequence model follows work introducing gated recurrent units in recurrent encoder-decoder learning [2]. The project also trains logistic regression as a simpler baseline. Comparing the two helps distinguish the contribution of sequence modeling from the existence of any predictive score.

Candidate-path routing reduces a large network decision to a small discrete action set. The project uses up to four loop-free paths ordered by base delay. The general problem of finding K shortest loopless paths was studied by Yen [3]. In the implementation, NetworkX provides a shortest-simple-path generator weighted by base delay; endpoints are sampled so the best two alternatives have limited link overlap. Thus actions have fixed meaning within an episode and have enough diversity for switching to be meaningful.

Several performance concepts are needed. PDR summarizes delivered traffic relative to offered demand. Mean path delay summarizes latency for traffic delivered. Recovery time measures how long a selected path remains outside the configured quality threshold after a failure. Proactive rate measures failures for which the policy has already left the affected path by the failure time. Route-switch count and unnecessary-switch rate

expose stability behavior. The predictor uses precision, recall, F1, PR-AUC, ROC-AUC, Brier score, and median lead time to characterize both ranking and thresholded detection.

1.4 Problem Statement

The project addresses the problem of selecting among a small number of candidate network paths when link quality is only partially observed and may degrade after the current observation. The controller sees noisy delay, loss, queue, utilization, and up/down measurements. Link health and future failure schedules remain hidden from the learned routing agent. At each decision point, the agent chooses one path and is evaluated on delivered traffic, delay, route stability, and recovery behavior.

The engineering challenge is to combine prediction and control without allowing the predictor to dominate current evidence or causing excessive route movement. A predictor can produce a risk estimate, but it does not itself select a route. The DQN must learn when the estimated risk justifies paying the cost of a switch. Reward weights, candidate-route quality, telemetry noise, failure timing, and the prediction horizon all affect this decision. The experiment therefore requires both controlled comparisons and explicit limits on what can be inferred from a simulated environment.

The project question is: under the tested simulator configurations, can a risk-aware DQN with a selected switch penalty reduce route changes relative to reactive and predictive baselines while maintaining comparable delivery and recovery outcomes? A related question is whether the link-risk predictor reaches the predeclared onset-detection target. The report answers these separately because routing stability can improve even when onset prediction is below target.

1.5 Gap Identification

The project-specific gap is the lack of a complete, controlled comparison showing how predicted risk and switching cost interact in the implemented routing environment. A reactive policy can show how DQN behaves with current telemetry. A predictive policy can show what changes when risk features are supplied. PRESTO-R can show the combined risk-plus-penalty policy. A shortest-path failover policy supplies a simple reference. The gap is especially important because prediction may prompt more switching, while a penalty may suppress both unnecessary and necessary switches.

The design addresses this gap by using a shared simulator, candidate-path interface, rollout code, and episode-level metric definitions. The comparison is repeated across two topologies and five scenario classes. It includes held-out seed matching, across-training-seed uncertainty, paired tests, predictor subgroup metrics, a lambda sweep, horizon sensitivity, and robustness checks. This structure allows the report to state where PRESTO-R reduces route churn, where PDR remains comparable, and where predefined criteria are not met.

The evidence leaves two material gaps open. First, the completed grid omits the no-prediction plus selected-penalty condition, so it does not form a full two-by-two factorial design. The available ablation cannot isolate a causal main effect for prediction and a causal main effect for switching cost. Second, the onset predictor misses its 0.70 F1 target. These are reported as limitations rather than filled with inferred values. Sudden-failure results also demonstrate that precursor-based prediction cannot warn about events that have no precursor.

This project does not claim to close a general scientific gap for all networks or reproduce an external RQSMR implementation. Its narrower contribution is a reproducible simulator-based evaluation of a specific risk-aware routing design, with negative and positive criteria reported together. This scope statement prevents simulated improvements from being presented as production-network evidence.

1.6 Objectives and Evaluation Alignment

The evaluation sheet groups the project into four criteria. Criterion A covers the problem statement, gap identification, and objectives. This report makes the routing-stability problem explicit, names the missing comparison, and ties each objective to a measurable outcome. Criterion B covers design methodology, innovation, and creativity. The design combines sequence-based risk, a shared path-feature state, DQN action values, a switch cost, and a held-out evaluation plan; the report describes this as a project-level design combination rather than a claim of unqualified algorithmic novelty.

Criterion C covers implementation, findings, and conclusion. The report details the simulator, telemetry, risk models, reward, training interface, evaluation pipeline, result tables, paired tests, robustness checks, limitations, and follow-up work. Criterion D covers presentation and project report. The document follows the supplied Times New

Roman and page-layout rules, includes front matter, numbered chapters, figures and tables with captions, references, appendices, and an evaluation evidence map. No marks are assigned in this report; the rubric values shown in the evaluation sheet are maximum weights, not scores.

Table 1.1 Evaluation Criteria And Evidence Map

Criterion	Maximum Weight	Report Evidence
A. Problem Statement, Gap Identification And Objectives	5	Chapter 1: problem, specific gap, objectives, scope, and evaluation alignment
B. Design Methodology, Innovation And Creativity	5	Chapter 4: architecture, methods, alternatives, trade-offs, and validity
C. Implementation, Findings And Conclusion	10	Chapters 3, 6, 7, and 8: specifications, implementation, complete reported outcomes, conclusions
D. Presentation And Project Report	10	Full document structure, formatting, figures, tables, citations, references, and reproducibility record

1.7 Scope and Contributions

The core scope includes implementation phases 0 through 6: repository setup, fluid simulator, failure schedules, telemetry and state construction, logistic regression and GRU predictors, heuristic baselines, reactive and predictive DQN policies, PRESTO-R, experiment grid, statistical reporting, figures, and local model packaging. The project includes NSFNET and RGG results, sensitivity sweeps, and three evaluation-only robustness variants. Optional NS-3 validation is outside the completed work.

The report contributes a traceable specification of the implemented system; an explicit statement of the research and evaluation gap; complete reporting of aggregate tables saved in the project outputs; comparison of positive and failed success criteria; and a list of limitations that constrain interpretation. It also preserves exact project metrics and distinguishes seed-level standard deviations from episode-level paired tests. These contributions support review and reproduction without implying production deployment.

CHAPTER 2
PROJECT DESCRIPTION AND GOALS

PRESTO-R is a predictive, stability-aware routing system implemented as an experimental pipeline rather than a deployable router. A seeded simulator generates a network episode, noisy link observations, and scheduled events. A risk provider estimates link degradation from telemetry. The observation builder aggregates measurements and risk into one row of path features per candidate. A policy selects one of four paths, receives reward and episode feedback, and writes step and event logs. Evaluation code transforms those logs into per-episode metrics, summaries, paired comparisons, figures, and a report. The following description separates project goals from achieved evidence and clarifies the boundary between simulation and live networking.

2.1 Project Overview

The project begins with topology and traffic initialization. NSFNET is represented by 14 nodes and 21 undirected links; RGG uses 24 nodes generated in a unit-square random geometric graph with connection radius 0.32, subject to connectivity and minimum-degree constraints. Each edge receives a stable link identifier, capacity, and base delay. The simulator creates background traffic, event schedules, and hidden health states. One managed flow is sampled with four sufficiently diverse routes.

After a ten-tick warm-up, the environment exposes measurements through a fixed-size observation. The default telemetry history is $W = 10$ ticks and the default prediction horizon is $H = 5$ ticks. A link-risk provider can supply zeros, an oracle upper bound for debugging, a GRU probability, or a logistic-regression probability. The deployed-in-simulation experiment uses learned risk for predictive policies. Oracle risk is not treated as a realistic operational method.

The routing policy chooses a candidate-path index. The simulator advances one tick, calculates true path quality, adds configured measurement noise to observations, updates telemetry, computes reward components, and records information. The policy is evaluated over a 200-tick measured episode following warm-up. Each run preserves resolved configuration and seed information so results can be tied to the exact model and settings.

2.2 Goals and Acceptance Criteria

The project goals are operationalized as measurable checks. The simulator must expose controlled scenarios with visible quality degradation; the predictor must produce calibrated risk estimates and be evaluated on onset samples; a reactive learned policy should be compared with shortest-path failover; PRESTO-R should reduce switches at comparable PDR; and a complete experiment grid must support statistical analysis and reproducible report generation.

The success record shows that PRESTO-R passed the comparisons against predictive DQN in mixed and fluctuation scenarios, passed the steady-state delay check, and passed the reproducibility check. It failed both specified comparisons against reactive DQN in node-failure and fluctuation scenarios, and failed the predictor onset-F1 target. These outcomes are kept visible in the report. Passing some checks does not change the failed ones or establish universal superiority.

A further project-level acceptance criterion is the availability of complete saved result artifacts. The test grid contains 38,400 episode metric rows, and the repository includes figures, summary tables, paired tests, robustness results, horizon and lambda sweeps, model weights, and a checksum manifest. The saved outputs support a comprehensive report, though the raw per-step logs are too granular to reproduce as thousands of pages in the main report.

Table 2.1 Project Goals And Measures

Goal	Measure Or Evidence	Recorded Outcome
Implement a controlled network simulator	Seeded episodes, two topologies, five scenarios	Implemented; full evaluation grid saved
Estimate future link risk	Onset F1, PR-AUC, calibration, lead time	GRU onset F1 0.5296; target 0.7000 not met
Reduce route churn with PRESTO-R	Switches per episode and PDR	Large switch reductions versus predictive DQN in mixed scenarios
Preserve delivery and recovery	PDR, delay, recovery time	PDR close to learned baselines; some reactive comparisons fail

Goal	Measure Or Evidence	Recorded Outcome
Support repeatable review	Seed separation, saved configs, hash verification	Step and event hashes match on a fresh rerun

2.3 System Boundary

The system boundary includes a Python fluid simulator, Gymnasium routing environment, data and predictor packages, reinforcement-learning policies, an experiment grid, statistical analysis, figure generation, and local artifact packaging. The input is a resolved configuration plus a deterministic seed stream. The outputs are episode actions, observed telemetry, event logs, metric tables, figures, and model artifacts.

The system does not include an IP routing protocol implementation, network device integration, live telemetry collectors, control-plane synchronization, multi-agent coordination, traffic engineering across multiple managed flows, or packet-level queue simulation. There is one managed flow per episode; other traffic is background and re-routed by the simulator. These boundaries are central when interpreting results and are repeated in the limitations chapter.

The optional NS-3 integration is a planned extension and is not part of the completed core workflow. No production network was accessed, and no live packets were routed by the learned agents. Therefore all evidence in this report is simulation evidence, including robustness conditions.

2.4 Actors and Operating Workflow

The policy is the decision-making actor. It observes only current measured path features, the current route indicator, a recency-since-switch feature, and, where configured, predicted risk. The simulator is the environment actor. It owns hidden link health, failure schedules, and true per-link state, but exposes only noisy observations to the policy. The evaluator reads rollout logs and ground truth after decisions have been made; it does not leak future information to the learned agent.

The predictor is trained as a separate supervised component. It uses episode-separated telemetry sequences and labels determined from future degradation within a horizon. Validation data selects a classification threshold. During evaluation, the predictor receives a rolling telemetry window and returns a probability per link. A risk

provider interface allows the environment to substitute learned risk, zero risk, or oracle risk for diagnostics.

The experiment runner enumerates method, topology, scenario, and seed combinations. It skips cells with existing logs, making the grid resumable. A metrics stage computes one row per episode. A statistical stage aggregates over training seeds, performs matched-seed tests, and applies Holm correction. Plot and report stages present aggregate results and illustrative traces. Each stage has a clear data contract so an evaluator can trace a result back to a run.

2.5 Assumptions

The model assumes a fixed candidate set within an episode and one selected path at a time. It assumes a discrete 100 ms time step, 200 evaluation ticks after warm-up, one managed flow, and a configured set of background flows. It assumes the observation fields can be normalized to a bounded feature vector and that risk scores are finite probabilities in [0,1].

The experiment assumes consistent metric definitions across methods, matched held-out scenario seeds where paired comparisons are possible, and no retuning on test data. The saved reports identify across-training-seed standard deviations for summary tables and use matched test-seed episode values for paired tests. Different result tables may therefore represent different aggregation layers.

The simulator uses synthetic event schedules, including precursor and sudden failures. A precursor model can only help if observable telemetry changes before a failure. The sudden-failure robustness condition deliberately removes precursor warning and is therefore a test of policy behavior when the predictor has no lead signal, not a fair prediction-accuracy test.

2.6 Evidence and Deliverables

The main code deliverables include modules for simulator dynamics, topology generation, traffic, failure schedules, telemetry, state construction, risk providers, reward, the Gymnasium environment, predictors, policies, training, rollout, metrics, statistics, figures, and report creation. Configuration files define base settings, topologies, scenarios, predictor variants, agent settings, and the evaluation grid.

The main evidence deliverables include 38,400 episode metric rows, topology-specific summary tables, paired-test tables, robustness data, horizon and lambda sweeps, predictor metrics for H5 and alternate horizons, nine result figures, a markdown report, and a local model-release archive with checksums. The report below reproduces all aggregate result rows described by these saved outputs. It does not inline every per-tick observation from the underlying 38,400 episodes.

CHAPTER 3
TECHNICAL SPECIFICATION

This chapter records the configuration and interfaces that determine how a run behaves. The project stores numerical settings in YAML and resolves them into run-specific configuration files. The simulator, predictor, agent, and evaluation grid share a common configuration system and seeded random streams. The specifications below distinguish values explicitly present in the repository from values that are not recorded. In particular, no specific host CPU model, memory size, GPU, or measured project cost is inferred.

3.1 Network and Simulator Specification

The simulator is a discrete-time fluid model. Each tick is 100 ms, each evaluation episode has 200 ticks after a warm-up of 10, and link capacity is 10.0 rate units. The NSFNET configuration samples base delays from 2 to 12 ms. The maximum queue is 20.0. Link capacity is multiplied by hidden health; a down link has zero effective capacity. Queue overflow, base loss, and degradation loss determine per-link losses, while delay includes a queue-dependent term and a timeout value for down links.

Background traffic is configured as 20 flows in the main mixed scenario, with log-normal rate parameters, a diurnal component, and small load noise. The managed flow rate is 2.0. It is routed over one of four paths. When links change availability, background traffic is re-routed. The model collects link load, utilization, delay, queue, loss, availability, and event records, then adds measurement noise to observable fields.

The simulator isolates random streams for topology, traffic, failures, and measurement noise. This prevents an extra noise draw from shifting a future failure schedule. Resolved configurations and seed identifiers are saved so a run can be reconstructed and comparisons can use aligned seeds.

3.2 Topology Specification

NSFNET is a fixed undirected graph with 14 nodes and 21 links. Link identifiers are assigned from sorted canonical endpoint pairs so that the feature arrays have stable indexing. Base delays are sampled within the configured interval and capacities are assigned from the topology configuration.

The RGG topology contains 24 nodes placed in a unit square. A random geometric graph connects nodes within radius 0.32. A generated topology is accepted only if it is connected and every node has degree at least two. Edge capacity is 10.0, queue maximum is 20.0, and base delay is 30 ms multiplied by Euclidean edge length. This produces a second topology with a different graph structure and path-length distribution.

For each episode, source and destination are selected with a seeded flow generator. Candidate paths are ordered by total base delay and must be loop-free. The first two paths are required to have limited overlap, providing meaningful alternatives. Up to four paths are selected and retained for the episode. The fixed action meanings simplify learning but do not model dynamic discovery of entirely new paths.

3.3 Traffic and Failure Specification

The scenario set contains steady, congestion, node_failure, fluctuation, and mixed conditions. The steady scenario schedules no congestion, link-failure, node-failure, or fluctuation events. The congestion scenario schedules an average of two congestion events per episode. The node-failure scenario uses link-failure and node-failure event rates of 0.5 and 1.0. The fluctuation scenario uses a rate of 1.5. The mixed scenario combines congestion, link failure, node failure, and fluctuation, with rates 1.0, 1.0, 0.3, and 0.5.

Mixed-scenario event targeting is configured to place an event on a candidate link with probability 0.7. Precursor duration ranges from 3 to 10 ticks, and 20 percent of applicable events may have no precursor. Down durations range from 15 to 60 ticks. Congestion multipliers range from 2.5 to 4.0 and durations from 10 to 40 ticks. Fluctuation periods range from 6 to 20 ticks with depth from 0.3 to 0.7. These values define simulator stress, not observed real-network frequency.

The sudden-failure robustness override sets no_precursor_prob to 1.0. The doubled-noise condition raises delay noise relative scale to 0.1, absolute loss noise to 0.01, and queue noise relative scale to 0.1. The 150 percent load condition increases background flow count from 20 to 30. These changes are evaluation-only; they are not used to retrain the agents or retune the selected penalty.

Table 3.1 Core Simulation Parameters

Parameter	Configured Value	Interpretation
Tick duration	100 ms	Discrete simulator time step
Warm-up	10 ticks	Fills initial observation history
Measured episode	200 ticks	Common evaluation horizon
Link capacity	10.0	Configured per-link capacity
Base delay, NSFNET	Uniform 2-12 ms	Per-link initial delay range
Queue maximum	20.0	Per-link queue cap
Candidate paths	4	Discrete action choices per episode
Telemetry window W	10 ticks	Historical link measurements
Default risk horizon H	5 ticks	Future interval for bad-link label
Selected switch penalty lambda	0.5	PRESTO-R reward configuration
Agent training steps	300,000	Configured training budget

Table 3.2 Scenario Configuration

Scenario	Event Configuration
Steady	No congestion, link failure, node failure, or fluctuation events.
Congestion	2.0 congestion events per episode on average.
Node failure	0.5 link-failure and 1.0 node-failure event rates.
Fluctuation	1.5 fluctuation events per episode on average.
Mixed	Congestion 1.0; link failure 1.0; node failure 0.3; fluctuation 0.5.

3.4 Telemetry and State Specification

The measured fields are delay, loss, queue, utilization, and an up flag. Delay noise is relative to delay or base delay; loss noise is absolute; queue noise is scaled by maximum queue; utilization has configured noise. Hidden link health, event schedules, and future degradation labels are not placed in the policy observation. The oracle risk provider can access future schedules only as a diagnostic upper bound and is not used as an ordinary learned policy input.

For each candidate path, the state builder computes normalized path delay, aggregate path loss, bottleneck queue, bottleneck utilization, normalized hop count, maximum predicted risk, mean predicted risk, and validity. When risk is disabled, the risk features are omitted. The observation appends a one-hot encoding of the current path

and a bounded time-since-switch feature. For four paths with risk, the observation dimension is 4 times 8 path features, plus 4 current-route indicators, plus one recency value, for 37 scalar inputs.

The state builder checks feature shapes, risk range, candidate-link identifiers, path index, reference delay, and maximum hops. Empty path slots are padded with zeros. Final values are clipped to [0,1] and emitted as float32. These checks protect the policy from invalid state shapes while keeping a fixed-size observation across episodes.

3.5 Predictor and Agent Specification

The main GRU predictor uses five telemetry features per link, a hidden size of 32, one recurrent layer, and dropout 0.1. The default training configuration uses a window of 10, horizon 5, bad-loss threshold 0.05, bad-delay factor 3.0, batch size 512, learning rate 0.001, up to 30 epochs, patience 5, and automatic positive-class weighting. Episode-level splits are 70 percent training, 15 percent validation, and 15 percent test, with negative subsampling configured at 0.3.

Logistic regression uses telemetry summaries and supplies a non-sequential predictor baseline. LearnedRisk loads a GRU checkpoint, normalizes input by saved means and standard deviations, and returns a probability per link. LogRegRisk loads a saved classifier and clips scores into [0,1]. Threshold selection maximizes validation F1; held-out test data is not used for threshold selection.

The routing agents use Stable-Baselines3 DQN. Reactive DQN disables risk features; predictive DQN enables risk but uses no selected switch penalty; PRESTO-R enables risk with lambda = 0.5. The same general training interface is used across methods. NSFNET uses training seeds 0 through 4, and RGG uses seeds 0 through 2. The best checkpoint is selected using validation reward.

3.6 Software Stack

The project pins Python packages in requirements.txt. The stack includes NumPy 1.26.4, pandas 2.2.2, SciPy 1.13.1, NetworkX 3.3, Gymnasium 1.0.0, Stable-Baselines3 2.4.0, sb3-contrib 2.4.0, PyTorch 2.4.1, scikit-learn 1.5.1, Matplotlib 3.9.2, PyYAML 6.0.2, tqdm 4.66.5, PyArrow 17.0.0, and pytest 8.3.2. The configured Python range is 3.10 through 3.12.

The module structure separates simulator code from the Gymnasium wrapper, predictors from routing policies, and rollouts from metrics. Configuration files reside under configs; source code under presto; scripts under scripts; tests under tests; results and figures under results; models under models; and saved training runs under runs. This separation supports small smoke executions as well as resumable large grids.

The project repository does not record a specific hardware inventory, total GPU hours, power draw, or labor cost. Training and full-grid evaluation runtime depend on the host, worker count, and storage. The report therefore lists observed computational volumes and software versions, while keeping unrecorded hardware and monetary amounts explicitly unavailable.

Table 3.3 Software Dependencies

Package	Version
Python	3.10-3.12
NumPy	1.26.4
pandas	2.2.2
SciPy	1.13.1
NetworkX	3.3
Gymnasium	1.0.0
Stable-Baselines3	2.4.0
PyTorch	2.4.1
scikit-learn	1.5.1
Matplotlib	3.9.2
PyArrow	17.0.0
pytest	8.3.2

3.7 Data and Reproducibility Specification

Training seeds are below 5000. Validation seeds are 5000 through 5049. Held-out test seeds are 10000 through 10199. The distinction is central: threshold and penalty selection use validation data; final test metrics are not used for tuning. Agent training seeds and scenario seeds are separate identifiers, and random streams isolate topology, traffic, failures, and noise.

The grid spans five scenarios and two topologies, with five learned-agent training seeds on NSFNET and three on RGG. The complete saved episode-metric table contains 38,400 rows. Step logs, event logs, resolved configurations, predictor metrics, summary tables, paired tests, robustness metrics, horizon tables, lambda sweep tables, and figures provide multiple review layers.

A fresh rerun produced matching hashes for saved step and event parquet outputs, according to the project success record. The model release bundle contains 88 curated files: four predictor checkpoints, 42 best agent policies, and their resolved configuration files. The local zip archive is 12.61 MB (13,227,437 bytes), with SHA-256 a0c3fcdd914f88ff2d899f4618fe9c4399ef3d3388806a067a5ea5dac4e1c398. Publishing to a remote host is not configured.

3.8 Metric Definitions

PDR is total delivered traffic divided by total offered demand over the episode. The mean delay is weighted by delivered traffic; it is not the unweighted mean of each path delay. Throughput is the mean delivered rate. Switches count actions that change the selected route. The unnecessary-switch rate counts switches for which the previous path remains healthy now and through the next configured horizon H.

A failure event is a true up-to-down transition on a link that was present on the selected path H ticks earlier. A proactive event is counted when the policy has already left the affected link by the failure time; such an event has zero-tick recovery. Otherwise recovery time is the first subsequent valid selected path with loss at or below 0.02, capped at the last recorded tick if no such recovery is observed. Some episodes have no qualifying events, so recovery time and proactive rate may be unavailable.

For predictor evaluation, precision is the share of predicted positive labels that are positive; recall is the share of actual positives detected; F1 is their harmonic mean. PR-AUC measures precision-recall ranking, ROC-AUC measures ranking across thresholds, and Brier score is mean squared probability error. Median lead time reports how far in advance a positive event is signaled under the project's definition. Each metric is reported with its evaluation subset and sample count.

$$\text{PDR} = \text{sum(delivered traffic)} / \text{sum(offered demand)} \tag{3.1}$$

In Eqn. 3.1, delivered traffic and offered demand are summed over measured episode ticks. PDR lies between zero and one for the recorded simulator outputs.

F1 = 2 x precision x recall / (precision + recall) (3.2)

In Eqn. 3.2, precision and recall are computed at a threshold selected on validation data. The report emphasizes onset F1 because this is the predeclared predictor success criterion.

Mean Delay = sum(path delay x delivered traffic) / sum(delivered traffic) (3.3)

In Eqn. 3.3, the weighting gives greater influence to delay experienced by delivered traffic and avoids treating fully lost traffic as ordinary successful delivery.

CHAPTER 4
DESIGN APPROACH AND DETAILS

The design separates five concerns: realistic-enough traffic and failure dynamics for controlled experiments; observation construction that hides future state; prediction from historical telemetry; route choice under an interpretable reward; and an evaluation process that separates tuning from final measurement. The system uses modular interfaces so the same simulator and rollout logic can compare heuristic, reactive, predictive, and stability-aware policies. The design does not depend on a production protocol stack and is not presented as one.

4.1 Design Approach Materials and Methods

The main design decision is to treat link-risk prediction as an additional observation feature and route switching as an explicit reward cost. This keeps the predictor and controller separable. The predictor emits per-link probabilities; the state builder aggregates them to maximum and mean risk for each candidate path. The DQN then learns whether those risk values, together with observed path quality, justify an action. This architecture supports ablations and diagnostics because risk can be disabled, learned, or replaced with oracle estimates without rewriting route-state logic.

The experimental method follows a staged workflow. First, simulator and environment behavior are validated. Second, telemetry is generated and predictor checkpoints are trained. Third, baselines and route agents are trained with multiple random seeds. Fourth, model checkpoints are selected on validation reward. Finally, the held-out grid is evaluated and summarized. The split prevents final test outcomes from influencing the selected threshold or switch cost.



Separate train, validation, and held-out test seeds; preserve resolved configuration and episode logs.

Fig. 4.1 PRESTO-R System Architecture

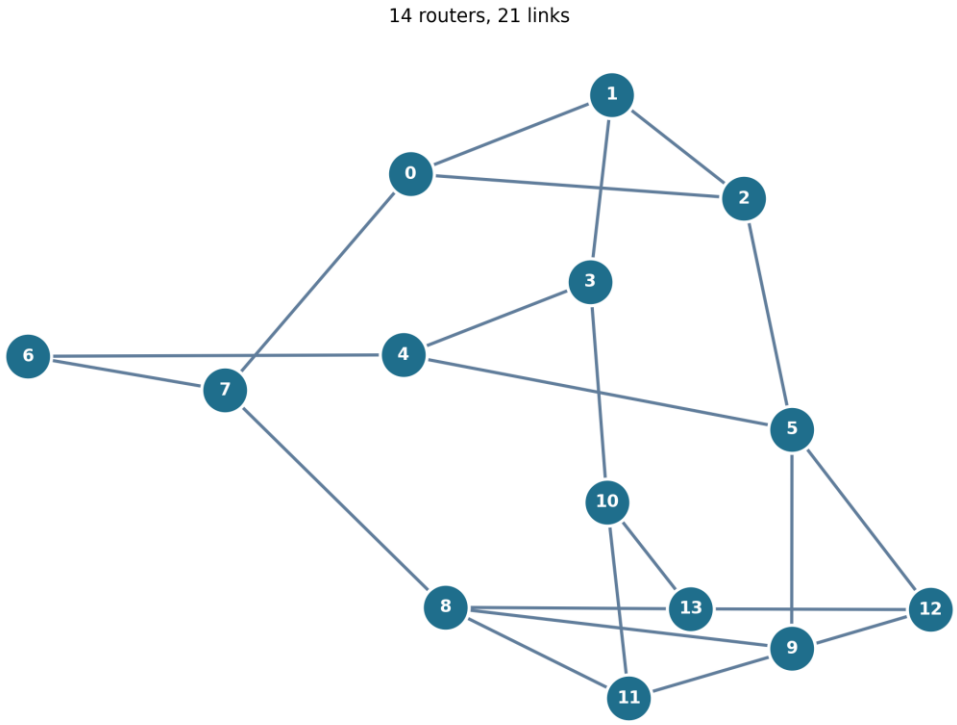


Fig. 4.2 NSFNET Topology Used In The Main Evaluation

Network Dynamics and Failure Model

The network simulator stores one state vector per link for capacity, base delay, queue, utilization, loss, and availability. At every tick it applies a precomputed event schedule to hidden health and availability, re-routes background flows when link-up state changes, adds the managed flow load to the chosen route, and updates queues from load minus effective capacity. Queue overflow contributes to loss. Delay increases with queue relative to effective capacity. A down link receives full loss and a timeout-based delay.

The fluid abstraction preserves causal relationships relevant to route choice while remaining computationally tractable for reinforcement learning. Capacity, delay, congestion, degradation, and failure are all configurable. The abstraction omits packet-level burst dynamics, TCP behavior, protocol convergence, route installation delay, and interactions among multiple controlled flows. These omissions make the test grid useful for controlled comparison, but not sufficient to certify a live deployment.

The event generator places congestion, link failures, node failures, and fluctuations according to scenario configuration. Some events include precursor behavior;

some are intentionally sudden. The simulator records true schedules for evaluation and labels, while the environment observation includes only noisy measurements. This separation makes risk prediction nontrivial and prevents a policy from reading future ground truth.

Candidate Path Selection and State Construction

Candidate routes are generated using shortest-simple-path ordering by base delay. A seeded source-destination flow is sampled repeatedly until four candidate routes are available and the best two do not share more than half the links of the shorter route. This diversity condition reduces the chance that all actions are effectively the same route. The candidates remain fixed in one episode so the action index has a stable interpretation.

The state builder aggregates link measurements across each path. Normalized delay is bounded relative to a reference path delay. Path loss is computed from component link losses. Queue and utilization features capture bottleneck conditions. Hop count is normalized by maximum candidate hops. Risk is represented by maximum and mean link risk. A binary validity flag indicates whether all links are up. The current path is encoded as one-hot, and elapsed ticks since the last switch are clipped and normalized.

The risk features are useful only if they carry information about future path quality. The project therefore compares reactive and predictive models and includes a risk-permutation diagnostic in the evaluation package. If permuting risk values did not affect behavior, that would suggest the policy was not using them. This diagnostic supports interpretation but does not eliminate the predictor's low onset-F1 limitation.

Telemetry Dataset and Label Design

Predictor training examples are generated from simulated episodes across congestion, node failure, fluctuation, and mixed scenarios and across NSFNET and RGG. Episodes are split before windows are formed, preventing adjacent windows from one episode from being scattered across training and test sets. The configured proportions are 70 percent training, 15 percent validation, and 15 percent test. Negative subsampling reduces the dominant healthy class during dataset construction.

A link is labeled bad when future simulator truth within the selected horizon meets the configured degradation conditions, including loss and delay thresholds or down state. The onset subset focuses on samples before a link is already bad, which is closer to

the early-warning task than classifying all time points. Threshold tuning maximizes F1 on validation data only. The held-out test split is then scored at that threshold.

The main horizon is H=5, with H=3 and H=10 studied separately. A longer horizon gives more lead time but asks the model to forecast farther ahead. That can increase ambiguity and lower onset classification quality. The report presents each horizon's predictor metrics alongside routing outcomes so model-level and policy-level performance are not conflated.

GRU and Logistic Regression Predictors

The GRU receives a time window for one link, with five telemetry features per time step. The recurrent layer summarizes sequence information in its hidden state. A linear head maps the final hidden state to a logit, and a sigmoid produces a bad-link probability. Input features are standardized using saved training means and standard deviations. Batched inference limits memory use during evaluation.

Logistic regression uses telemetry summaries rather than a recurrent hidden state. It offers a simpler comparison with lower temporal modeling capacity. Both models produce one probability per link. Scenario, topology, and event-type subgroups expose variation hidden by an overall score. The precursor and sudden-failure rows are particularly informative: sudden events lack predictive warning by design, while precursor events contain signals that could be learned.

The main report shows high all-sample scores for both predictors, but all-sample scores are not the primary criterion. GRU onset F1 is 0.5296, and logistic-regression onset F1 is 0.3949. The F1 target is 0.7000. The gap between overall and onset results shows why subgroup reporting is necessary. Calibration scores and ranking metrics provide additional context but do not substitute for the missed onset target.

DQN Policies and Reward Design

The action space contains one discrete action per candidate path. DQN learns an action-value function from state and reward interactions. The reactive agent sees current path features without risk. The predictive DQN sees risk features but uses no selected penalty. PRESTO-R uses risk features and lambda=0.5. The saved trained policies are evaluated using deterministic rollouts on held-out scenarios.

The reward is the sum of interpretable terms. Delay is normalized relative to a reference path and capped. Loss is penalized strongly. Throughput can be weighted, though the main configured gamma is zero. A switch incurs lambda. Hop count contributes a small energy or path-length term. In the core configuration, alpha=1.0, beta=4.0, gamma=0.0, lambda=0.5 for PRESTO-R, and mu=0.05. The reward exposes each component in the environment info record, supporting diagnostics of why a policy action receives its value.

A switch cost introduces a direct stability preference. At a high value, it can suppress a beneficial path change. At a low value, the policy may react frequently to small fluctuations. The project sweeps five lambda values on held-out mixed scenarios after selecting the main configuration on validation. The sweep is reported as an evaluation of sensitivity, not as permission to choose a new value using test data.

$$R = -\alpha D - \beta L + \gamma T - \lambda S - \mu E \quad (4.1)$$

In Eqn. 4.1, D is normalized delay, L is path loss, T is normalized throughput, S is a binary switch indicator, and E is normalized hop count. The configured PRESTO-R coefficients are alpha=1.0, beta=4.0, gamma=0.0, lambda=0.5, and mu=0.05.

Rollout, Metric, and Statistical Method

Each rollout records per-step policy choice, path links, observed telemetry, risk values, true quality values, demand, delivered rate, reward, and scenario seed. Event logs record scheduled event types and timings. The metrics stage groups rows by episode and computes PDR, delay, throughput, switches, unnecessary switches, recovery time, proactive rate, energy, reward, failure events, and scheduled events.

Summary tables first average episode metrics within a training seed, then average those seed-level means across training seeds. Standard deviations describe training-seed variation. Paired tests use matched held-out scenario seeds and Wilcoxon signed-rank comparisons. Holm correction adjusts across the global comparison family. Effect size r is signed according to the median difference. Recovery time only has pairs for episodes with relevant events, so its sample count is often below the PDR or switch sample count.

The summary table and paired test table answer different questions. Mean +/- SD communicates variability across trained policies. A paired test evaluates matched test-

seed episode differences and yields its own median difference and adjusted p-value. The report keeps both views separate and does not infer a paired statistical conclusion from overlapping standard deviations.

Robustness and Sensitivity Design

The robustness tests modify one evaluation condition while keeping the trained checkpoint. Sudden failure removes warning. Doubled measurement noise changes the observation quality. A 50 percent increase in background flows raises offered load and congestion pressure. These variants determine whether the qualitative stability effect survives changes to event warning, observation quality, or background load.

The horizon sweep uses H=3 and H=10 PRESTO-R variants on mixed and node-failure scenarios. It reports PDR, switches, proactive rate, recovery time, and predictor onset F1. The lambda sweep uses values 0.0, 0.05, 0.1, 0.25, and 0.5 for mixed scenarios. Together they expose trade-offs: more penalty tends to reduce switches, while longer forecast horizon can improve lead time but weaken onset classification.

The no-prediction plus selected-penalty condition was not included in the completed test grid. The report therefore describes the available partial ablation and explicitly avoids calling it a complete factorial experiment. This is essential to the causal interpretation of the combined PRESTO-R design.

4.2 Codes and Standards

The project uses the DQN family for discrete path selection and uses GRU and logistic-regression models for risk estimation. The DQN approach is related to Mnih et al. [1]; the gated recurrent design is related to Cho et al. [2]; and the k-shortest loopless path concept is related to Yen [3]. These citations establish methodological lineage and do not imply that the project reproduces those papers' full experiments.

The software dependency versions are pinned in requirements.txt. Configuration-driven values, separated seeds, resolved configs, hash manifests, and deterministic evaluation outputs support computational reproducibility. The repository includes tests for topology, telemetry, simulator behavior, state construction, reward, predictor, metrics, rollout, reporting, and experiment phases. The saved success record specifically reports a fresh-rerun hash match for step and event parquet outputs.

No external network protocol standard such as OSPF, IS-IS, BGP, MPLS, or an IEEE forwarding standard is implemented by the simulation. The project abstracts path selection at the candidate-route level. This boundary is stated to avoid presenting an algorithmic simulator as protocol-compliant software. The report follows the page, typography, caption, and numbering conventions shown in the supplied VIT project-report sample.

Table 4.1 Design Alternatives And Trade-Offs

Design Choice	Alternative	Reason For Choice	Consequence
Fluid network simulation	Packet-level network simulation	Allows large RL training and held-out grids on a CPU	Less packet and protocol fidelity
One managed flow	Many controlled flows	Keeps the discrete action and state tractable	No inter-agent coordination evidence
Four fixed paths per episode	Dynamic route discovery every step	Stable action meanings and compact policy state	Cannot learn dynamic candidate generation
GRU predictor plus logistic baseline	Single predictor only	Tests sequential model against a simpler classifier	Both learn simulator-derived patterns
DQN policies	PPO or rule-only control	Discrete action set and shared training interface	No algorithm-family comparison in final grid
Switch penalty lambda=0.5	No cost or alternate penalty only	Validation-screened configuration with test sweep	Full no-prediction plus penalty factorial is missing

4.3 Constraints Alternatives and Tradeoffs

The fluid model trades packet fidelity for throughput of experiment. It represents congestion and queue effects, but it does not model individual packets, TCP congestion control, real routing protocols, or deployment overhead. Results are therefore evidence about the defined simulator only. A packet-level follow-up should preserve event and metric semantics where possible so model rankings can be compared.

One managed flow provides a tractable action space but leaves multi-flow conflicts untested. Background flows create shared load, yet they are not strategic agents.

A multi-flow extension would need coordination, per-flow fairness, and an expanded reward. It could change both the value of a route switch and the learned risk response.

Fixed candidate routes make DQN action semantics stable. A dynamic path generator could offer alternatives when a route fails, but would add state churn and variable action meaning. The fixed set can also become stale if topology changes substantially. This is a deliberate initial trade-off, not an accidental omission.

A GRU and logistic-regression baseline enable a sequential-versus-summary comparison. Both operate on synthetic telemetry generated by the same simulator family. Transfer to real traffic is unknown. The onset sample imbalance is an analytical limitation: high all-sample F1 does not mean reliable precursor detection. Better event-balanced training, calibrated subgroup thresholds, or domain-randomized data could be investigated, but must be evaluated on held-out scenarios.

The selected switch cost reduces route changes, but a penalty can delay a necessary switch. The mixed and fluctuation results show strong switching reductions versus predictive DQN. Comparisons with reactive DQN fail some predeclared checks, and the full factorial to isolate component effects is incomplete. A future experiment should include every prediction-by-penalty combination and pre-register the paired criteria before test evaluation.

4.4 Statistical Design and Validity

The experimental grid uses held-out scenario seeds from 10000 to 10199. NSFNET learned methods use five training seeds, while RGG learned methods use three. Seed-level means are used to report variability across training runs. Paired tests match scenario seeds between PRESTO-R and baselines, drop unavailable paired metrics, and adjust p-values with Holm correction. These choices reduce the risk of treating repeated episodes from one trained policy as independent training replications.

The sample count in paired PDR and switch rows is generally 200 episodes. Recovery time counts only episodes containing the relevant events, giving smaller n values. A median difference of zero may coexist with a small p-value when a large number of matched episodes contain many tied values plus a few nonzero ranks; interpretation must retain effect size and median difference, not just p-value.

External validity is limited by one managed flow, synthetic event schedules, simulator-derived telemetry, a fluid traffic model, and a finite topology set. Construct validity depends on metric definitions such as the horizon used to classify unnecessary switches and the 0.02 loss quality threshold for recovery. Reproducibility evidence is positive for saved step and event hashes, but it does not establish external validity.

CHAPTER 5
SCHEDULE, TASKS AND MILESTONES

The project plan is organized into core phases 0 through 6 and an optional Phase 7. The phase durations below are planning estimates recorded in the implementation plan, not reconstructed calendar times. The repository indicates that the core simulator, predictors, routing agents, full experiment grid, figures, report artifacts, and model package are present. Actual start and finish dates for each phase are not logged in the material reviewed, so the report does not invent them.

5.1 Work Breakdown

Phase 0 established the repository, environment, configuration system, and CI-style checks. Phase 1 implemented the network simulator, failure scenarios, telemetry, state, reward, and Gymnasium environment. Phase 2 created the link-degradation dataset, logistic regression and GRU models, risk interfaces, and evaluation. Phase 3 implemented heuristic baselines, deterministic rollouts, and reactive DQN. Phase 4 added predictive and PRESTO-R policies, penalty screening, ablations, and diagnostics.

Phase 5 built the complete experiment grid, episode metrics, statistical tests, robustness checks, horizon sweep, lambda sweep, and success criteria. Phase 6 generated figures and report artifacts, documented reproduction, and packaged models with a checksum manifest. Phase 7 proposed optional NS-3 validation after the core work; that work is not included in the completed project evidence.

The workflow dependencies are sequential where simulator contracts are required, while predictor and reactive-agent work can proceed in parallel after the environment is stable. Experiment grid generation depends on trained models and resolved configurations. Report generation depends on saved raw results and derived tables. The phase structure supports incremental validation and makes it possible to identify which component would need revision if a metric or policy behavior changed.

Table 5.1 Phase Schedule And Completion Status

Phase	Planned Work	Estimated Effort	Status In Repository
0	Repository, environment, config, checks	0.5-1 day	Core workflow present

Phase	Planned Work	Estimated Effort	Status In Repository
1	Simulator, failures, telemetry, state, reward, environment	4-5 days	Implemented
2	Dataset, logistic regression, GRU, risk providers	2-3 days	Implemented; saved metrics available
3	Heuristics, rollouts, reactive DQN	2-3 days	Implemented
4	PRESTO-R, lambda tuning, ablations, diagnostics	2-3 days	Implemented with partial factorial gap
5	Experiment grid, metrics, statistics, sweeps	2-3 days	Completed; 38,400 episode rows
6	Figures, report, reproduction and model release	1-2 days	Artifacts present
7	Optional NS-3/ns3-gym validation	3+ days	Optional; not included

5.2 Phase Schedule and Status

The phase map lists about 14 to 20 working days for the core path, excluding the optional NS-3 phase. These values are estimated effort rather than actual elapsed time. The repository does not include a dated work log that would support a claim about calendar completion. A formal academic schedule should therefore use the planned effort as a baseline and not present it as recorded duration.

The setup phase is evidenced by pinned dependencies, project configuration, and a Makefile. Phase 1 is evidenced by simulator and environment modules and tests. Phase 2 is evidenced by dataset, training, evaluation, trace, model checkpoints, and predictor metric outputs. Phases 3 and 4 are evidenced by agents, policies, training runs, switch-penalty variants, and diagnostics. Phase 5 is evidenced by a full grid and derived tables. Phase 6 is evidenced by generated figures, a project report, and the local model archive.

Phase 7 remains optional. No NS-3 validation results are cited in this report. The absence of this phase does not mean the Python simulator was validated against packet-level output. It marks a known next step.

5.3 Milestone Evidence

The simulator milestone is supported by discrete-time dynamics, event schedules, noisy observations, and episode state. The predictor milestone is supported by saved

GRU and logistic regression metrics across subgroups and horizons. The routing milestone is supported by reactive, predictive, PRESTO-R, and heuristic policies. The evaluation milestone is supported by topology-specific result tables, paired tests, robustness outcomes, and success criteria.

The report milestone is supported by a generated markdown report, figures, and an export script. The release milestone is supported by an 88-file curated bundle and a SHA-256 manifest. The reproducibility milestone is supported by the recorded fresh-rerun hash match. These evidence items verify that artifacts exist; they do not independently verify every result from an external clean-room environment.

5.4 Reproducibility and Release

The README gives commands for environment setup, smoke checks, predictor data creation and training, agent training, grid evaluation, metrics, statistics, success checks, figures, report generation, and model packaging. The experiment grid is resumable and skips completed cells. Resolved configurations accompany runs so metric values can be tied to exact parameters.

The model package contains predictor checkpoints, best agent policies, and resolved configuration files. The manifest records SHA-256 digests. The archive size is 12.61 MB and its exact byte size is 13,227,437. There is no configured remote release destination. Model publishing is therefore local-only in the observed workspace.

CHAPTER 6
PROJECT DEMONSTRATION

The demonstration is a reproducible software workflow rather than a live network console. It begins with a configuration and seed, produces a simulated episode, updates telemetry and risk, selects a candidate path, and records observations and outcomes. Evaluation commands then transform saved episode logs into statistics and visualizations. The code excerpts below are selected from implemented modules and are included to make the central mechanics reviewable; they are not placeholders or pseudo-code.

6.1 Sample Codes

The simulator and environment are separate layers. The simulator advances network dynamics and returns per-link statistics. The environment exposes a Gymnasium observation and action space, calls the risk provider, computes reward, and packages diagnostic information. This separation allows a heuristic policy and a learned policy to share the same simulated network and metrics.

The observation builder is intentionally fixed-size. It validates input shapes, encodes path measurements, appends optional risk and current-route indicators, and clips the resulting vector to the agent's declared bounds. The reward function returns both a scalar training signal and a dictionary of component values, which supports analysis of delay, loss, throughput, switching, and hop contributions.

Metric computation occurs after rollout. It uses true schedules and event records to determine failures and recovery, while the policy only receives the noisy observations. This distinction keeps evaluation truth out of the learning interface. The following excerpt shows the reward decomposition as implemented in presto/reward.py.

Source Excerpt 6.1 Reward Components - presto/reward.py

```
"""Routing reward and its interpretable component terms."""

from dataclasses import dataclass


@dataclass
class RewardWeights:
    alpha: float = 1.0
    beta: float = 4.0
```

```
    gamma: float = 0.0
    lam: float = 0.1
    mu: float = 0.05


def compute_reward(
    qos: dict,
    switched: bool,
    ref_delay: float,
    max_hops: int,
    demand: float,
    w: RewardWeights,
) -> tuple[float, dict[str, float]]:
    """Score one action using normalized delay/loss, optional throughput, switching and
hops."""
    if ref_delay <= 0 or max_hops < 1 or demand <= 0:
        raise ValueError("ref_delay, max_hops, and demand must be positive")
    delay_term = min(float(qos["delay"]) / ref_delay, 3.0) / 3.0
    loss_term = float(qos["loss"])
    throughput_term = float(qos["delivered"]) / demand
    switch_term = float(switched)
    energy_term = float(qos["hops"]) / max_hops
    terms = {
        "D": -w.alpha * delay_term,
        "L": -w.beta * loss_term,
        "T": w.gamma * throughput_term,
        "S": -w.lam * switch_term,
        "E": -w.mu * energy_term,
    }
    reward = terms["D"] + terms["L"] + terms["T"] + terms["S"] + terms["E"]
    return float(reward), terms
```

The returned terms are D, L, T, S, and E, corresponding to delay, loss, throughput, switching, and hop cost. The method validates positive normalization constants before calculating reward. This design makes a switch penalty inspectable in logs and allows the same reward structure to be used across agent configurations.

Observation Construction

The state builder represents each route with normalized path delay, path loss, bottleneck queue, bottleneck utilization, hop count, optional maximum and average link risk, and validity. It pads missing routes with zeros, adds a one-hot current-route vector, and includes time since switch. Because the same feature layout is used at every step, the DQN can compare route values consistently.

The main implementation validates candidate paths and risk arrays before encoding. It rejects invalid link indices, bad feature dimensions, nonpositive reference delay, and out-of-range current path values. Those checks convert silent state corruption into explicit errors, which is important when the grid contains thousands of episodes.

Source Excerpt 6.2 Path Feature Definitions - presto/state.py

```
"""Candidate-path selection and normalized agent observations."""

from math import prod

import networkx as nx
import numpy as np

PATH_FEATURES = [
    "delay_norm",
    "loss",
    "bneck_queue",
    "bneck_util",
    "hops_norm",
    "max_risk",
    "mean_risk",
    "valid",
]


def candidate_paths(
    graph: nx.Graph, src: int, dst: int, k: int, lid: dict[tuple[int, int], int]
) -> list[list[int]]:
    """Get up to k loop-free paths, ordered by total base delay."""
    if src == dst or k < 1:
        raise ValueError("Candidate paths require distinct endpoints and k >= 1")
```

Episode Evaluation

Episode metrics are computed from saved step rows and event truth. A switch is unnecessary if the old path remains healthy for the current and next H ticks. Failure events are true up-to-down transitions on a path selected H ticks earlier. If the link is no longer on the selected path at failure time, the event is proactive and recovery is counted as zero ticks. Otherwise the first valid selected path at or below the QoS loss threshold determines recovery.

The metric implementation weights mean delay by delivered traffic, divides total delivery by total demand for PDR, and counts switches directly from the recorded action

transitions. When event truth is unavailable, recorded down-link information is used as a documented fallback. The complete function is long, so the excerpt below preserves the key metric rules in source form.

Source Excerpt 6.3 Metric Contract - presto/eval/metrics.py

```
    """Compute delivery, switching, recovery, proactive, energy and reward metrics.

    A switch is unnecessary if its old path is healthy now and throughout the
    next H ticks. Failure events are true up-to-down transitions whose link was
    on the selected path H ticks earlier. Proactive events have already left
    that link by failure time and count as zero-tick recoveries. Otherwise,
    recovery is the first valid selected path with loss <= qos_ok_loss, capped
    at the last recorded episode tick. If exact ``sim_bad`` truth is omitted,
    rollout ``true_bad_links`` is used; only schedule ``sim_up`` or recorded
    down links are used as a fallback.
    """
    required = {
        "t",
        "switched",
        "path_links",
        "path_loss",
        "path_delay",
        "delivered",
        "demand",
        "hops",
        "reward",
    }
    missing = required - set(steps.columns)
    if missing:
        raise ValueError(f"steps is missing required columns: {',
'.join(sorted(missing))}")
    if H < 1 or not 0.0 <= qos_ok_loss <= 1.0:
        raise ValueError("H must be positive and qos_ok_loss must be in [0, 1]")
    if steps.empty:
        raise ValueError("steps must contain at least one episode row")
    if steps["scenario_seed"].nunique() > 1:
        raise ValueError("episode_metrics accepts one episode at a time")

    steps = steps.sort_values("t").reset_index(drop=True)
    up, bad = _truth_arrays(steps, sim_up, sim_bad)
    tick_rows = {int(row.t): row for row in steps.itertuples(index=False)}
    switches = steps[steps["switched"].astype(bool)]
    unnecessary = 0
    for row in switches.itertuples(index=False):
        tick = int(row.t)
```

```
        if hasattr(row, "previous_path_links"):
            old_path = _links(row.previous_path_links)
        else:
            prior_rows = steps[steps["t"] < tick]
            old_path = _links(prior_rows.iloc[-1]["path_links"]) if not prior_rows.empty
    else []
        if not old_path:
            continue
```

6.2 Sample Screenshots

The project stores analysis visualizations as PNG and PDF images. These figures are generated from saved simulator runs and evaluation summaries. They are included here as project-output screenshots, not screenshots of a live production network. Each figure caption identifies what the plot shows and whether it is aggregate or illustrative.

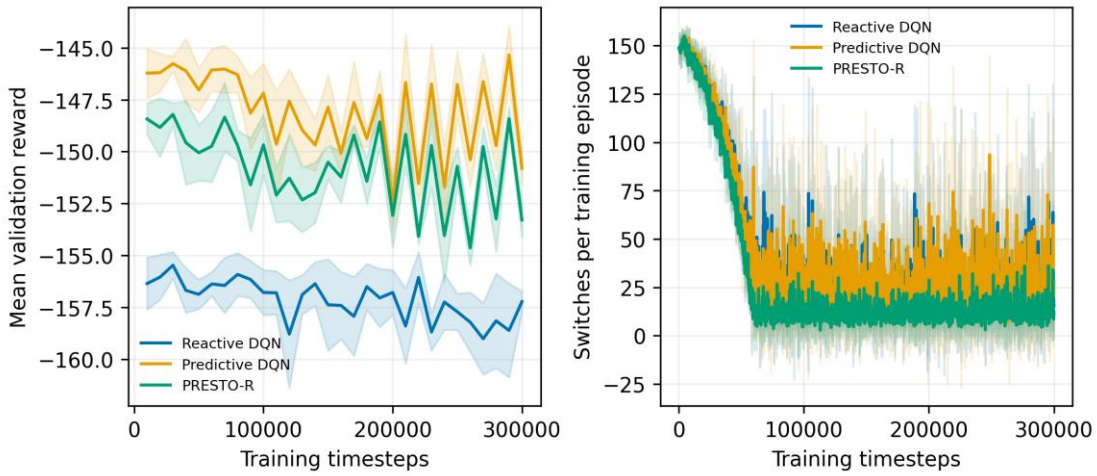


Fig. 6.1 Representative Training Curves

The learning-curve figure displays training reward and switching behavior for the learned methods. It provides training context but does not replace held-out test results. Training curves can indicate whether reward changes over the training budget; final comparisons rely on the held-out evaluation grid and its seed-aware summaries.

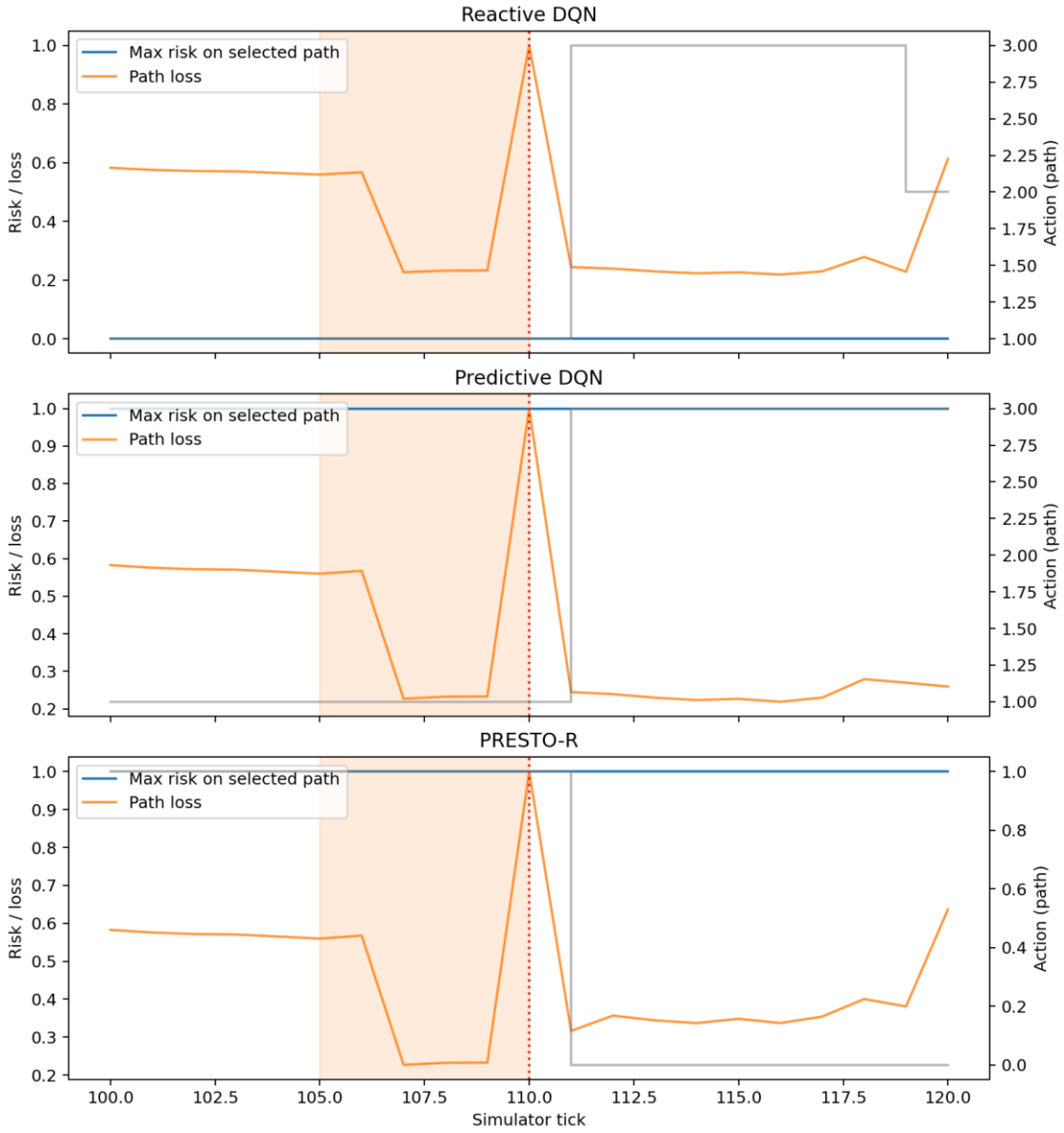


Fig. 6.2 Held-Out Event Trace Demonstration

The event trace is selected from the first suitable held-out mixed episode among the first 20 test seeds. It illustrates model and policy behavior around a particular event. It is not a representative average and should be read with the per-episode statistics and scenario summaries.

6.3 End-to-End Demonstration

The documented workflow first installs the pinned requirements and runs the smoke validation. It then creates episode-separated predictor data, trains and evaluates the GRU and logistic baseline, trains the reactive, predictive, and PRESTO-R agents, and

executes the experiment grid. The metrics command generates the episode parquet table; the statistics command builds paired and grouped summaries; the success checker records predeclared outcomes; plot and report commands generate the final figures and report.

The principal output locations are configs for parameterization, runs for trained agents and logs, data for predictor sequences, results/raw for step and event logs, results/tables for derived metrics, results/figures for plots, models for release artifacts, and results/report.md plus results/report.pdf for report outputs. The experiment grid can resume by skipping cells with saved logs.

A reproducibility review should use the resolved configuration associated with each run, the seed identifiers, and the exact model checkpoint. The project's checksum manifest applies to packaged model files. It does not hash every report image or every raw episode file. The reported fresh-rerun hash match specifically applies to step and event parquet outputs noted in the success criteria.

CHAPTER 7
COST ANALYSIS RESULT AND DISCUSSION

This chapter presents every aggregate result recorded in the project report, predictor metrics, summary tables, paired-test table, horizon sweep, lambda sweep, robustness data, and predeclared success record. Mean values are kept with their reported dispersion or paired-test statistics. “n/a” indicates that the source table did not report a value or that the episode did not contain an event needed to calculate it. The report does not convert unavailable values to zero or suppress failed comparisons.

7.1 Cost and Resource Analysis

The repository does not record monetary expenditures, hourly labor, host hardware specifications, electricity use, or cloud billing. No dollar or rupee total can be reported without inventing data. The cost analysis therefore documents measurable computational assets and the information gap for monetary costing.

The configured model training budget is 300,000 steps per main agent run. NSFNET learned methods use five agent-training seeds and RGG methods use three. The held-out grid contains 38,400 episode metric rows. The predictor uses up to 30 epochs, batch size 512, and early stopping patience 5. These values characterize computational scale, but do not by themselves determine runtime or hardware cost.

The local model archive is 12.61 MB (13,227,437 bytes), contains 88 curated files, and is accompanied by a checksum manifest. The project uses pinned open-source Python dependencies listed in Chapter 3. No remote release destination is configured. A future operational cost study would need measured training and evaluation time, worker count, storage use, hardware model, electricity rate, and personnel hours.

Table 7.1 Observed Computational Assets And Unrecorded Cost Items

Category	Recorded Evidence	Monetary Cost
Training	300,000 configured agent steps; multiple seeds	Not recorded
Evaluation	38,400 episode metric rows; five scenarios; two topologies	Not recorded

Category	Recorded Evidence	Monetary Cost
Software	Pinned open-source Python package versions	No license payment recorded in project files
Model package	88 curated files; 12.61 MB archive	Not recorded
Hardware	Host CPU/GPU/RAM inventory absent	Not recorded
Labor	Phase effort estimates exist; actual hours absent	Not recorded
External publishing	No remote release destination configured	No publishing charge recorded

7.2 Evaluation Protocol

The principal grid compares sp_failover, reactive_dqn, predictive_dqn, and presto_dqn across five scenarios on NSFNET and RGG. Main NSFNET learned agents have five training seeds; RGG agents have three. Each summary cell averages episode-level metrics within each training seed, then reports mean and standard deviation across seed means. The deterministic shortest-path heuristic has no between-training-seed deviation, so its standard deviation is unavailable.

The paired comparison table evaluates PRESTO-R against reactive and predictive DQN on matched NSFNET held-out seeds. It reports episode count, median target-minus-baseline difference, Holm-adjusted p-value, and effect-size r. The paired tests include PDR, switches, and recovery time where available. The table's episode count for recovery may be substantially smaller because only event-bearing episodes contribute.

The main risk horizon is five ticks. Horizon-specific variants use three and ten ticks. Lambda screening considers zero, 0.05, 0.1, 0.25, and 0.5. Robustness cases are sudden failure, twice the measurement noise, and 150 percent background load. Robustness uses existing policies and evaluation overrides rather than retraining on altered test conditions.

7.3 Predictor Results

The main held-out predictor results distinguish all-sample classification from the onset subset. The onset subset is central because it excludes links already bad at the prediction time. The all-sample GRU F1 of 0.9769 is high, but onset F1 is 0.5296. The

validation-selected onset threshold is 0.816824. The project's target is 0.7000, so the primary predictor criterion fails.

Scenario subgroups show stronger GRU onset performance in fluctuation than in node failure, while topology subgroups show different precision-recall trade-offs. Event-type subgroups show high precursor PR-AUC but lower Brier calibration and smaller sample sizes; the sudden-failure group cannot be interpreted as predictive early warning because by design it contains no precursor. Logistic regression has lower overall onset F1 and lower onset PR-AUC than the GRU in the main H5 test metrics.

The full metric tables include threshold, precision, recall, F1, PR-AUC, ROC-AUC, Brier score, lead time where calculated, and sample count. H3 and H10 predictor subgroups are also included. This prevents the report from hiding subgroup behavior behind one average.

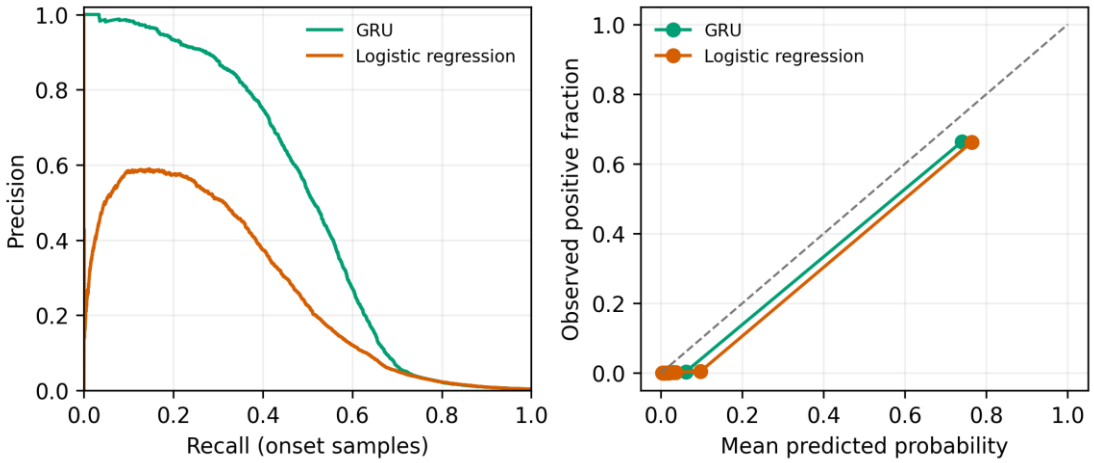


Fig. 7.1 Held-Out Predictor Quality

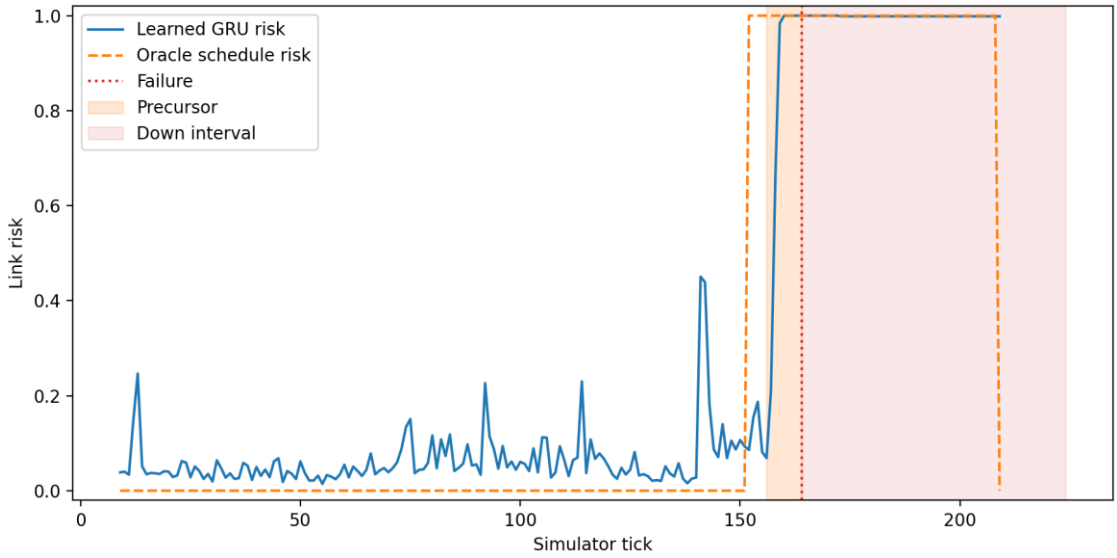


Fig. 7.2 Learned And Oracle Link-Risk Trace

Table 7.2 GRU Test Classification And Sample Counts

Subset	Group	Threshold	Precision	Recall	F1	Samples
all	all	0.896057	0.988647	0.965497	0.976935	822705
onset	all	0.816824	0.649811	0.446864	0.529559	769718
onset	scenario:congestion	0.835795	0.602597	0.401384	0.481828	194086
onset	scenario:fluctuation	0.874231	0.748062	0.580451	0.653683	191727
onset	scenario:mixed	0.812896	0.660177	0.428736	0.519861	177123
onset	scenario:node_failure	0.749318	0.618399	0.376556	0.468085	206782
onset	topology:nsfnet	0.778013	0.614173	0.464286	0.528814	166112
onset	topology:rgg	0.868345	0.708029	0.416607	0.524561	603606
onset	failure:precursor	0.00661398	0.627346	0.924901	0.747604	1256
onset	failure:sudden	0.00460567	0.5	1	0.666667	240

Table 7.3 GRU Test Ranking Calibration And Lead Time

Subset	Group	PR-AUC	ROC-AUC	Brier	Median Lead	Samples
all	all	0.986083	0.994259	0.00542893	3	822705

Subset	Group	PR-AUC	ROC-AUC	Brier	Median Lead	Samples
onset	all	0.49455	0.896925	0.005364	3	769718
onset	scenario:congestion	0.448079	0.955789	0.00521983	n/a	194086
onset	scenario:fluctuation	0.664936	0.940421	0.0049552	n/a	191727
onset	scenario:mixed	0.48465	0.897464	0.00584858	n/a	177123
onset	scenario:node_failure	0.413798	0.833071	0.00546328	n/a	206782
onset	topology:nsfnet	0.502654	0.883856	0.0124368	n/a	166112
onset	topology:rgg	0.487595	0.87644	0.00341756	n/a	603606
onset	failure:precursor	0.863948	0.766711	0.334316	n/a	1256
onset	failure:sudden	0.538833	0.497014	0.4615	n/a	240

Table 7.4 LOGREG Test Classification And Sample Counts

Subset	Group	Threshold	Precision	Recall	F1	Samples
all	all	0.777368	0.983837	0.956426	0.969938	822705
onset	all	0.605217	0.434308	0.362041	0.394895	769718
onset	scenario:congestion	0.679206	0.365535	0.242215	0.291363	194086
onset	scenario:fluctuation	0.64126	0.531409	0.470677	0.499203	191727
onset	scenario:mixed	0.586094	0.43843	0.372414	0.402735	177123
onset	scenario:node_failure	0.563913	0.406417	0.315353	0.35514	206782
onset	topology:nsfnet	0.605217	0.436831	0.364286	0.397274	166112
onset	topology:rgg	0.583853	0.408805	0.372226	0.389659	603606
onset	failure:precursor	0.00320672	0.608766	0.988142	0.75339	1256
onset	failure:sudden	0.00295987	0.502165	0.966667	0.660969	240

Table 7.5 LOGREG Test Ranking Calibration And Lead Time

Subset	Group	PR-AUC	ROC-AUC	Brier	Median Lead	Samples
all	all	0.983634	0.994146	0.00750375	1	822705
onset	all	0.265605	0.895756	0.00751876	2	769718
onset	scenario:congestion	0.184012	0.960255	0.00733569	n/a	194086
onset	scenario:fluctuation	0.38116	0.940063	0.00685125	n/a	191727
onset	scenario:mixed	0.262947	0.898823	0.00836169	n/a	177123
onset	scenario:node_failure	0.250896	0.82602	0.00758746	n/a	206782
onset	topology:nsfnet	0.26663	0.878331	0.0175279	n/a	166112
onset	topology:rgg	0.271206	0.876543	0.00476424	n/a	603606
onset	failure:precursor	0.802175	0.742821	0.356644	n/a	1256
onset	failure:sudden	0.534775	0.500208	0.451633	n/a	240

Table 7.6 GRU H=3 Classification And Sample Counts

Subset	Group	Threshold	Precision	Recall	F1	Samples
all	all	0.904112	0.992322	0.98258	0.987427	842569
onset	all	0.770519	0.677786	0.536937	0.599196	789588
onset	scenario:congestion	0.748322	0.630631	0.47138	0.539499	191128
onset	scenario:fluctuation	0.818788	0.663202	0.718468	0.68973	194369
onset	scenario:mixed	0.770519	0.676768	0.462069	0.54918	196258
onset	scenario:node_failure	0.74393	0.784768	0.484663	0.599241	207833
onset	topology:nsfnet	0.74393	0.658143	0.530945	0.58774	168629
onset	topology:rgg	0.787586	0.69697	0.556452	0.618834	620959
onset	failure:precursor	0.0890594	0.660194	0.786127	0.717678	846
onset	failure:sudden	0.00153655	0.497608	1	0.664537	209

Table 7.7 GRU H=3 Ranking Calibration And Lead Time

Subset	Group	PR-AUC	ROC-AUC	Brier	Median Lead	Samples
all	all	0.993953	0.997594	0.00260271	2	842569
onset	all	0.566664	0.922241	0.00241153	2	789588
onset	scenario:congestion	0.514499	0.964962	0.00210825	n/a	191128
onset	scenario:fluctuation	0.696833	0.945396	0.00271021	n/a	194369
onset	scenario:mixed	0.477943	0.889795	0.002512	n/a	196258
onset	scenario:node_failure	0.566672	0.902642	0.00231623	n/a	207833
onset	topology:nsfnet	0.533133	0.913206	0.00595987	n/a	168629
onset	topology:rgg	0.606559	0.903278	0.00144793	n/a	620959
onset	failure:precursor	0.838724	0.842561	0.162611	n/a	846
onset	failure:sudden	0.557149	0.521978	0.481495	n/a	209

Table 7.8 GRU H=10 Classification And Sample Counts

Subset	Group	Threshold	Precision	Recall	F1	Samples
all	all	0.930543	0.987488	0.935933	0.961019	813200
onset	all	0.763284	0.508523	0.442732	0.473352	759188
onset	scenario:congestion	0.748499	0.434519	0.509317	0.468954	193674
onset	scenario:fluctuation	0.80277	0.586555	0.578773	0.582638	190413
onset	scenario:mixed	0.746304	0.486231	0.353567	0.40942	178540
onset	scenario:node_failure	0.712409	0.505314	0.394717	0.44322	196561
onset	topology:nsfnet	0.802551	0.544034	0.41919	0.473522	161720
onset	topology:rgg	0.762356	0.51476	0.443738	0.476618	597468
onset	failure:precursor	0.00364994	0.788536	1	0.881767	1291

Subset	Group	Threshold	Precision	Recall	F1	Samples
onset	failure:sudden	0.0132311	0.507812	1	0.673575	512

Table 7.9 GRU H=10 Ranking Calibration And Lead Time

Subset	Group	PR-AUC	ROC-AUC	Brier	Median Lead	Samples
all	all	0.973434	0.988482	0.0110637	5	813200
onset	all	0.42868	0.871449	0.0114808	8	759188
onset	scenario:congestion	0.472722	0.969814	0.0108886	n/a	193674
onset	scenario:fluctuation	0.5519	0.891792	0.0110522	n/a	190413
onset	scenario:mixed	0.353879	0.836156	0.0134528	n/a	178540
onset	scenario:node_failure	0.371198	0.812057	0.0106881	n/a	196561
onset	topology:nsfnet	0.439267	0.863066	0.0259606	n/a	161720
onset	topology:rgg	0.417776	0.841261	0.00756142	n/a	597468
onset	failure:precursor	0.890672	0.662835	0.541288	n/a	1291
onset	failure:sudden	0.491397	0.480433	0.461763	n/a	512

7.4 NSFNET Results

The NSFNET summary includes 20 method-scenario rows. Across all five scenarios, PRESTO-R makes fewer switches per episode than both learned comparators. In mixed episodes, PRESTO-R averages 5.197 switches versus 20.988 for predictive DQN and 18.799 for reactive DQN, while PDR is 0.9625 versus 0.9608 and 0.9619. The 75.2 percent switch reduction relative to predictive DQN is consistent with the intended stability effect.

PDR remains high for learned policies across scenarios, but the improvements over reactive DQN are not sufficient to claim every planned criterion passes. In node failure, PRESTO-R PDR is 0.9667 versus 0.9662 for reactive DQN; the success record nevertheless marks the comparison as failed because the paired criterion also considers recovery and corrected statistical results. In fluctuation, PRESTO-R PDR is 0.9749

versus 0.9740, but recovery comparison and multiplicity-adjusted tests again prevent a pass.

The shortest-path failover policy offers a clear non-learning reference, with substantially lower PDR and much higher delay. Its switch count is near zero because the heuristic does not behave like the learned route selectors. A low switch count alone is not a good outcome when delivery is poor. Therefore the main interpretation considers PDR, delay, recovery, switching, proactive response, and unnecessary switching together.

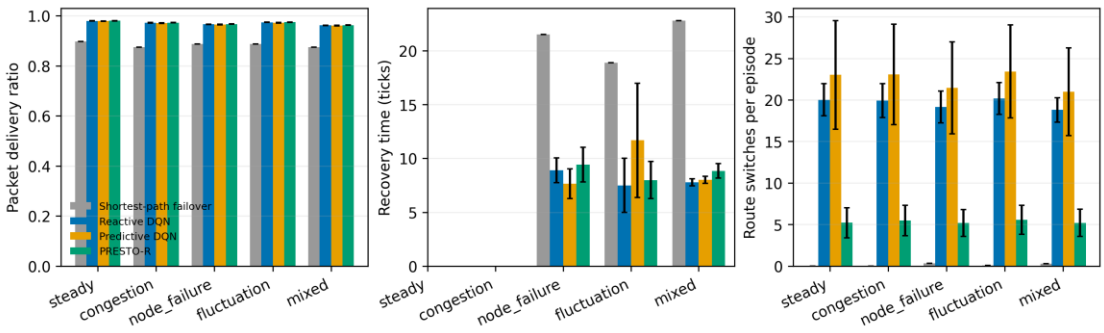


Fig. 7.3 NSFNET Main Results

Table 7.10 NSFNET Delivery Delay And Recovery (Mean +/- Standard Deviation Across Training Seeds)

Method	Scenario	Pdr	Delay Mean	Recovery Time
sp_failover	steady	0.8966 ± NA	119.9542 ± NA	not available
sp_failover	congestion	0.8747 ± NA	128.1184 ± NA	not available
sp_failover	node_failure	0.8871 ± NA	117.6611 ± NA	21.5272 ± NA
sp_failover	fluctuation	0.8875 ± NA	124.2878 ± NA	18.8750 ± NA
sp_failover	mixed	0.8751 ± NA	126.5853 ± NA	22.8085 ± NA
reactive_dqn	steady	0.9797 ± 0.0009	46.3062 ± 0.7003	not available
reactive_dqn	congestion	0.9721 ± 0.0009	52.6087 ± 0.6362	not available
reactive_dqn	node_failure	0.9662 ± 0.0010	52.6682 ± 0.8414	8.8951 ± 1.1546
reactive_dqn	fluctuation	0.9740 ± 0.0011	49.7406 ± 0.6025	7.4906 ± 2.5128
reactive_dqn	mixed	0.9619 ± 0.0008	56.6613 ± 0.7133	7.7741 ± 0.3258
predictive_dqn	steady	0.9786 ± 0.0008	46.7965 ± 1.5823	not available

Method	Scenario	Pdr	Delay Mean	Recovery Time
predictive_dqn	congestion	0.9708 ± 0.0010	53.0682 ± 1.4765	not available
predictive_dqn	node_failure	0.9652 ± 0.0013	53.4841 ± 1.3807	7.6455 ± 1.3806
predictive_dqn	fluctuation	0.9726 ± 0.0011	50.7261 ± 1.6293	11.6746 ± 5.3045
predictive_dqn	mixed	0.9608 ± 0.0013	57.1976 ± 0.9776	8.0037 ± 0.3307
presto_dqn	steady	0.9804 ± 0.0007	46.2766 ± 1.0454	not available
presto_dqn	congestion	0.9729 ± 0.0006	52.4105 ± 0.8563	not available
presto_dqn	node_failure	0.9667 ± 0.0007	52.5669 ± 0.8969	9.4236 ± 1.5982
presto_dqn	fluctuation	0.9749 ± 0.0007	49.8912 ± 1.1183	7.9852 ± 1.7305
presto_dqn	mixed	0.9625 ± 0.0005	56.3287 ± 0.9610	8.8419 ± 0.6810

Table 7.11 NSFNET Switching Proactive Response And Unnecessary Switches (Mean +/- Standard Deviation Across Training Seeds)

Method	Scenario	Switches	Proactive Rate	Unnecessary Switch Rate
sp_failover	steady	0.0000 ± NA	not available	not available
sp_failover	congestion	0.0000 ± NA	not available	not available
sp_failover	node_failure	0.3100 ± NA	0.0068 ± NA	0.0000 ± NA
sp_failover	fluctuation	0.0700 ± NA	0.0000 ± NA	0.0000 ± NA
sp_failover	mixed	0.2650 ± NA	0.0000 ± NA	0.0000 ± NA
reactive_dqn	steady	20.0070 ± 1.9388	not available	0.4127 ± 0.0349
reactive_dqn	congestion	19.9130 ± 2.0395	not available	0.3818 ± 0.0225
reactive_dqn	node_failure	19.1490 ± 1.9134	0.5070 ± 0.0331	0.3675 ± 0.0284
reactive_dqn	fluctuation	20.1760 ± 1.9200	0.2886 ± 0.0470	0.3907 ± 0.0245
reactive_dqn	mixed	18.7990 ± 1.4575	0.4572 ± 0.0339	0.3444 ± 0.0279
predictive_dqn	steady	23.0240 ± 6.5355	not available	0.4975 ± 0.0390

Method	Scenario	Switches	Proactive Rate	Unnecessary Switch Rate
predictive_dqn	congestion	23.0640 ± 6.0548	not available	0.4524 ± 0.0247
predictive_dqn	node_failure	21.4570 ± 5.5372	0.5872 ± 0.0444	0.4429 ± 0.0200
predictive_dqn	fluctuation	23.4190 ± 5.6024	0.2294 ± 0.0928	0.4656 ± 0.0215
predictive_dqn	mixed	20.9880 ± 5.2802	0.5108 ± 0.0578	0.4128 ± 0.0342
presto_dqn	steady	5.2120 ± 1.8193	not available	0.2836 ± 0.0585
presto_dqn	congestion	5.4780 ± 1.8468	not available	0.2794 ± 0.0614
presto_dqn	node_failure	5.1820 ± 1.6214	0.4472 ± 0.0426	0.2412 ± 0.0500
presto_dqn	fluctuation	5.5570 ± 1.7518	0.2418 ± 0.0564	0.2766 ± 0.0575
presto_dqn	mixed	5.1970 ± 1.6276	0.3901 ± 0.0524	0.2344 ± 0.0494

NSFNET Scenario Analysis: Steady

In steady NSFNET episodes the shortest-path baseline has PDR 0.8966 and mean delay 119.9542. Reactive DQN reaches PDR 0.9797 with delay 46.3062 and 20.007 switches. Predictive DQN reaches PDR 0.9786, delay 46.7965, and 23.024 switches. PRESTO-R reaches PDR 0.9804, delay 46.2766, and 5.212 switches. Thus the switch penalty sharply reduces route changes while PDR and delay remain near the learned baselines.

No recovery time or proactive rate is available for the learned steady episodes because there are no failure events that qualify for those measures. The heuristic has zero switches in steady state, but its delivery and delay are substantially worse than the learned policies. This demonstrates why route stability cannot be judged without delivery quality.

NSFNET Scenario Analysis: Congestion

Under NSFNET congestion, the heuristic PDR is 0.8747 and delay is 128.1184. Reactive DQN records PDR 0.9721, delay 52.6087, and 19.913 switches. Predictive DQN records PDR 0.9708, delay 53.0682, and 23.064 switches. PRESTO-R records PDR 0.9729, delay 52.4105, and 5.478 switches. Relative to predictive DQN, PRESTO-R has slightly higher PDR and lower delay while making fewer switches.

The paired tests show a statistically significant but numerically very small PDR median difference against predictive DQN, and a substantial switch-count difference against both predictive and reactive baselines. The key interpretation is the magnitude distinction: a p-value can be small because 200 matched episodes are available even when the PDR median difference is tiny. Route-switch reductions are much larger in absolute units.

NSFNET Scenario Analysis: Node Failure

In NSFNET node-failure episodes, PDR is 0.9662 for reactive DQN, 0.9652 for predictive DQN, and 0.9667 for PRESTO-R. Mean delays are 52.6682, 53.4841, and 52.5669 respectively. Mean recovery times in the training-seed summaries are 8.8951, 7.6455, and 9.4236. PRESTO-R switches 5.182 times, versus 19.149 for reactive DQN and 21.457 for predictive DQN.

The result shows the central trade-off. PRESTO-R reduces switching and maintains PDR near the reactive policy, but does not deliver faster average recovery in the summary table. The success criteria mark the comparison against reactive DQN as failed. The predictor's onset subgroup for node failure also has lower F1 than the fluctuation subgroup. Failure management must therefore be judged by recovery and detection, not by switch reduction alone.

NSFNET Scenario Analysis: Fluctuation

In NSFNET fluctuation episodes, PDR is 0.9740 for reactive DQN, 0.9726 for predictive DQN, and 0.9749 for PRESTO-R. Mean delay is 49.7406, 50.7261, and 49.8912. PRESTO-R switches 5.557 times per episode, compared with 20.176 for reactive DQN and 23.419 for predictive DQN. PRESTO-R has fewer switches and slightly higher PDR than either learned comparator, while its recovery-time summary is 7.9852 versus 7.4906 for reactive DQN.

The project success record marks PRESTO-R versus predictive DQN as passed on the switch/PDR criterion and marks PRESTO-R versus reactive DQN as failed under the predeclared comparison. Paired tests report corrected PDR p-values and a zero median difference against reactive DQN despite the summary PDR mean difference. These statistics are not interchangeable; both are reported.

NSFNET Scenario Analysis: Mixed

Mixed NSFNET episodes combine congestion, link failure, node failure, and fluctuation. The summary PDR is 0.9619 for reactive DQN, 0.9608 for predictive DQN, and 0.9625 for PRESTO-R. Mean delays are 56.6613, 57.1976, and 56.3287. Recovery times are 7.7741, 8.0037, and 8.8419. Switch counts are 18.799, 20.988, and 5.197. PRESTO-R therefore reduces switches by 75.2 percent versus predictive DQN, while its PDR is higher by 0.0017.

The proactive rate is 0.3901 for PRESTO-R, compared with 0.4572 for reactive DQN and 0.5108 for predictive DQN. Unnecessary-switch rate is 0.2344 for PRESTO-R, lower than 0.3444 and 0.4128 for the two comparators. These measures suggest the penalty curbs unnecessary route movement but also changes the policy's proactive behavior. The missing factorial cell prevents isolating how much of this behavior is caused by risk features versus the penalty.

7.5 Paired Tests

Paired comparisons use common held-out scenario seeds and report the median difference PRESTO-R minus baseline. Negative switch differences favor PRESTO-R because fewer switches are desired. Positive PDR differences favor PRESTO-R. Holm-adjusted p-values control the family-wise error rate across the specified comparisons. Effect size r provides direction and magnitude information under the saved analysis convention.

Across scenarios, switch comparisons against predictive and reactive DQN are strongly significant with negative effects, reflecting fewer route changes under PRESTO-R. PDR comparisons generally have small median differences, even where adjusted p-values are small. Recovery-time tests use only failure episodes, so n varies. Several recovery comparisons have zero median difference and nonsignificant p-values; the node-failure comparison with predictive DQN has a corrected p of 0.0493 and an effect size of zero in the saved table, so it should not be described as a large recovery effect.

All paired-test rows from the saved NSFNET table are reproduced below, including zero differences and nonsignificant results. This avoids selecting only favorable tests. The summary tables remain separately available for across-training-seed means and standard deviations.

Table 7.12 Paired Wilcoxon Results With Holm Correction

Scenario	Metric	Baseline	Episodes	Median Difference	p Holm
congestion	pdr	predictive_dqn	200	3.989e-06	8.57e-10
congestion	pdr	reactive_dqn	200	0	0.914
congestion	switches	predictive_dqn	200	-7.8	8.38e-22
congestion	switches	reactive_dqn	200	-4.7	1.63e-20
fluctuation	pdr	predictive_dqn	200	2.4945e-06	1.47e-06
fluctuation	pdr	reactive_dqn	200	0	0.0104
fluctuation	recovery_time	predictive_dqn	16	0	1
fluctuation	recovery_time	reactive_dqn	18	0	1
fluctuation	switches	predictive_dqn	200	-8.9	8.86e-23
fluctuation	switches	reactive_dqn	200	-4.1	3.05e-21
mixed	pdr	predictive_dqn	200	2.302e-05	4.77e-05
mixed	pdr	reactive_dqn	200	0	0.616
mixed	recovery_time	predictive_dqn	58	0	1
mixed	recovery_time	reactive_dqn	59	0	0.472
mixed	switches	predictive_dqn	200	-8.1	1.2e-25
mixed	switches	reactive_dqn	200	-6.3	1.15e-23
node_failure	pdr	predictive_dqn	200	5.4865e-06	0.00154
node_failure	pdr	reactive_dqn	200	0	1
node_failure	recovery_time	predictive_dqn	53	0	0.0493
node_failure	recovery_time	reactive_dqn	51	0	0.262
node_failure	switches	predictive_dqn	200	-8	2.02e-23
node_failure	switches	reactive_dqn	200	-6.2	3.41e-22

Scenario	Metric	Baseline	Episodes	Median Difference	p Holm
steady	pdr	predictive_dqn	200	0	3.52e-08
steady	pdr	reactive_dqn	200	0	0.341
steady	switches	predictive_dqn	200	-4.1	9.87e-19
steady	switches	reactive_dqn	200	-2.2	2.21e-17

Table 7.13 Paired-Test Signed Effect Sizes

Scenario	Metric	Baseline	Effect Size r
congestion	pdr	predictive_dqn	0.482
congestion	pdr	reactive_dqn	0.000
congestion	switches	predictive_dqn	-0.713
congestion	switches	reactive_dqn	-0.691
fluctuation	pdr	predictive_dqn	0.399
fluctuation	pdr	reactive_dqn	0.000
fluctuation	recovery_time	predictive_dqn	0.000
fluctuation	recovery_time	reactive_dqn	0.000
fluctuation	switches	predictive_dqn	-0.728
fluctuation	switches	reactive_dqn	-0.704
mixed	pdr	predictive_dqn	0.354
mixed	pdr	reactive_dqn	0.000
mixed	recovery_time	predictive_dqn	0.000
mixed	recovery_time	reactive_dqn	0.000
mixed	switches	predictive_dqn	-0.772
mixed	switches	reactive_dqn	-0.742
node_failure	pdr	predictive_dqn	0.301
node_failure	pdr	reactive_dqn	0.000
node_failure	recovery_time	predictive_dqn	0.000
node_failure	recovery_time	reactive_dqn	0.000
node_failure	switches	predictive_dqn	-0.739
node_failure	switches	reactive_dqn	-0.719

Scenario	Metric	Baseline	Effect Size r
steady	pdr	predictive_dqn	0.000
steady	pdr	reactive_dqn	0.000
steady	switches	predictive_dqn	-0.661
steady	switches	reactive_dqn	-0.637

7.6 Ablation and Penalty Sweep

The available mixed-scenario ablation cells compare no prediction/no penalty, prediction/no penalty, and prediction/selected penalty. Reactive DQN has PDR 0.9619 and 18.799 switches. The zero-penalty predictive variant has PDR 0.9628 and 17.812 switches. The selected-penalty PRESTO-R cell has PDR 0.9625 and 5.197 switches. The no-prediction/selected-penalty cell was not run in the completed P0 grid.

The lambda sweep shows average mixed-scenario PDR staying within a narrow band as switch rate falls from 17.8125 at lambda 0.0 to 5.3625 at lambda 0.5. Unnecessary-switch rate decreases through lambda 0.25 and rises slightly at 0.5. Recovery and proactive metrics vary substantially across episodes and their standard deviations are large. The sweep illustrates a stability-delivery trade-off but does not identify a universal optimum.

Because the no-prediction plus penalty condition is absent, the effect of prediction cannot be separated from the effect of the switch penalty in a complete factorial analysis. The report treats the partial cells descriptively and avoids a causal decomposition.

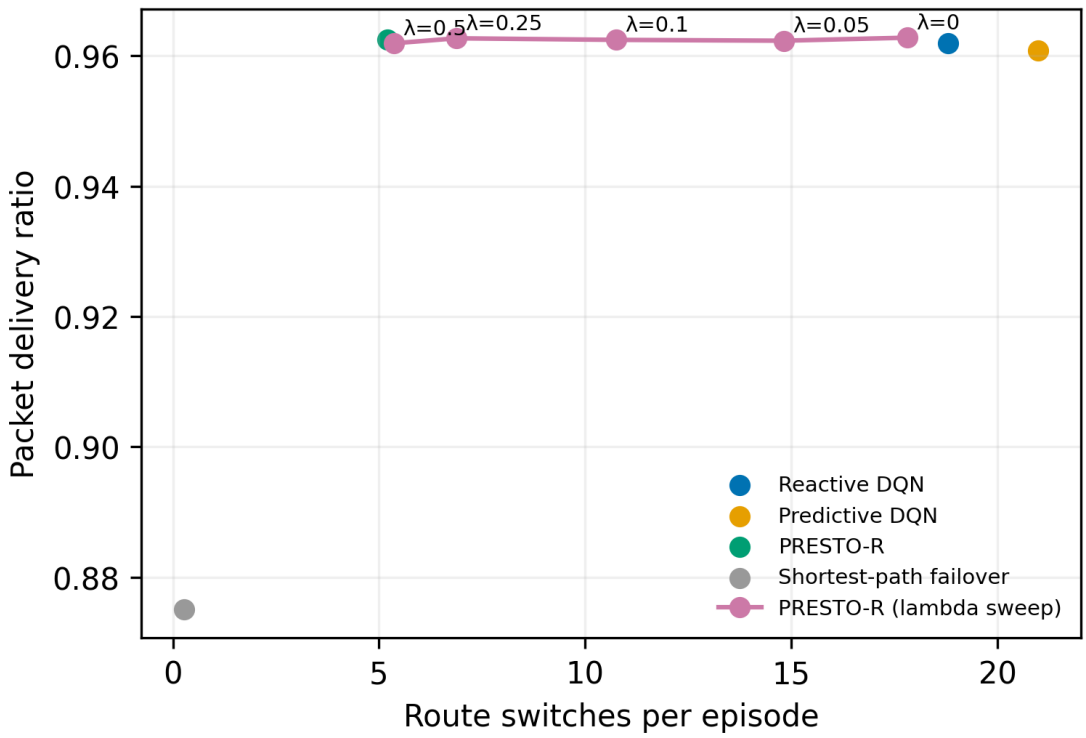


Fig. 7.4 Mixed-Scenario Switch-Penalty Trade-Off

Table 7.14 Available Ablation Cells

Ablation	Method	PDR Mean	Switches Mean
No prediction; no penalty	reactive_dqn	0.9619	18.799
Prediction; no penalty	presto_lam0	0.9628	17.812
Prediction; selected penalty	presto_dqn	0.9625	5.197
No prediction; selected penalty	reactive_dqn_pen	not run in the completed P0 grid	not run in the completed P0 grid

Table 7.15 Lambda Sweep PDR And Switches

Lambda	Method	Pdr Mean	Pdr Std	Switches Mean	Switches Std
0.0	presto_lam0	0.9628	0.0697	17.8125	25.7984
0.05	presto_lam005	0.9623	0.071	14.8125	24.591
0.1	presto_lam01	0.9624	0.0705	10.7475	17.7226
0.25	presto_lam025	0.9627	0.0703	6.8725	12.9127

Lambda	Method	Pdr Mean	Pdr Std	Switches Mean	Switches Std
0.5	presto_lam05	0.9619	0.0725	5.3625	9.5133

Table 7.16 Lambda Sweep Unnecessary Switching And Recovery

Lambda	Method	Unnecessary Switch Rate Mean	Unnecessary Switch Rate Std	Recovery Time Mean	Recovery Time Std
0.0	presto_lam0	0.376	0.3446	7.5033	12.9154
0.05	presto_lam005	0.3562	0.3407	9.2273	14.1703
0.1	presto_lam01	0.3167	0.3147	8.5316	13.8551
0.25	presto_lam025	0.2486	0.288	9.3152	14.5388
0.5	presto_lam05	0.2585	0.2963	8.6821	13.8873

Table 7.17 Lambda Sweep Proactive Rate

Lambda	Method	Proactive Rate Mean	Proactive Rate Std
0.0	presto_lam0	0.5248	0.4769
0.05	presto_lam005	0.4327	0.4773
0.1	presto_lam01	0.4509	0.4819
0.25	presto_lam025	0.4239	0.4723
0.5	presto_lam05	0.378	0.4662

7.7 Horizon Sensitivity

The H=3 mixed result has PDR 0.9613, 6.8733 switches, proactive rate 0.3463, recovery time 9.7458, and onset F1 0.5992. H=10 mixed has PDR 0.9616, 5.5583 switches, proactive rate 0.3912, recovery time 8.8698, and onset F1 0.4734. On node failure, H=3 has PDR 0.9661 and H=10 has 0.9658; switch means are 6.6750 and 5.8983, with F1 of 0.5992 and 0.4734.

The longer horizon yields fewer switches in both scenarios and similar PDR, but onset F1 is lower. This result illustrates why horizon selection cannot rely only on routing stability. A model that forecasts farther ahead may produce behavior that changes route decisions, yet has a weaker onset classifier. Recovery means and proactive rates also vary by scenario and include their reported across-seed standard deviations.

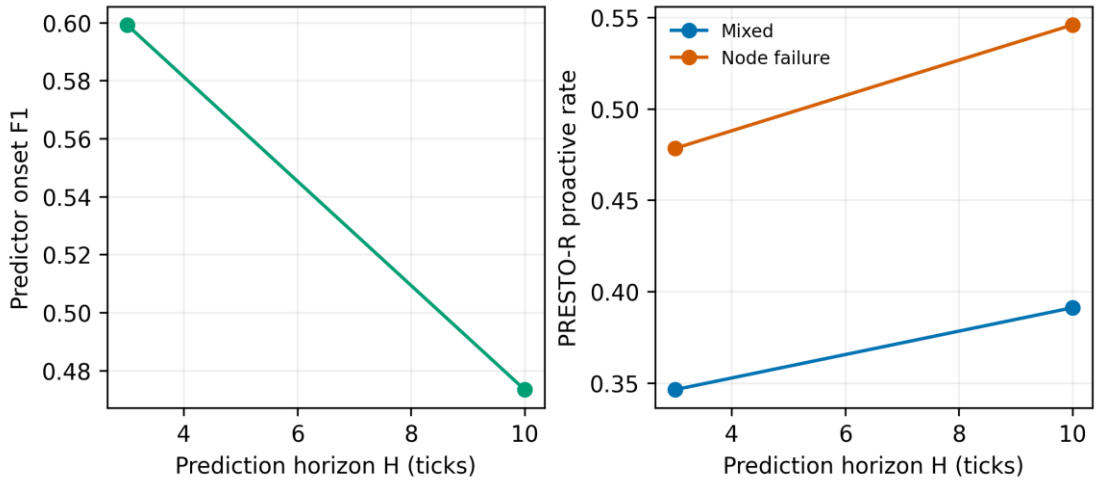


Fig. 7.5 Prediction-Horizon Sensitivity

Table 7.18 Horizon Sweep PDR And Switching

Horizon	Scenario	Pdr Mean	Pdr Std	Switches Mean	Switches Std
3	mixed	0.9613	0.0016	6.8733	2.0496
3	node_failure	0.9661	0.0017	6.675	1.9609
10	mixed	0.9616	0.0015	5.5583	1.8525
10	node_failure	0.9658	0.0019	5.8983	2.2085

Table 7.19 Horizon Sweep Proactivity Recovery And Predictor F1

Horizon	Scenario	Proactive Rate Mean	Proactive Rate Std	Recovery Time Mean	Recovery Time Std	Predictor Onset F1
3	mixed	0.3463	0.0653	9.7458	0.3708	0.5992
3	node_failure	0.4783	0.0352	9.3308	1.8138	0.5992
10	mixed	0.3912	0.0171	8.8698	0.4399	0.4734
10	node_failure	0.5459	0.0096	9.2278	1.1084	0.4734

7.8 RGG Results

The RGG topology is a second structural check with 24 nodes and a random geometric link layout. Main learned policies use three training seeds. In mixed episodes, predictive DQN reaches PDR 0.9546, delay 38.4670, recovery time 17.1224, and 11.873 switches. PRESTO-R reaches PDR 0.9545, delay 38.1368, recovery 16.3996, and 1.913 switches. The switch reduction is large while PDR differs by only 0.0001.

In steady episodes, PRESTO-R has PDR 0.9669 and 1.0383 switches, while reactive DQN has 0.9665 and 12.4383 switches. In congestion, PRESTO-R records PDR 0.9627, delay 35.5245, and 1.3033 switches. Node failure and mixed scenarios show longer recoveries on RGG than NSFNET, reflecting different topology and event exposure. The topology-specific values should not be compared as if only the policy changed; network structure also changes.

Across all five RGG scenarios, PRESTO-R reduces route switches relative to both learned baselines. PDR is close to the predictive policy. The RGG results strengthen evidence that the stability effect is not limited to one topology, while the small number of training seeds and simulated setting still limit generalization.

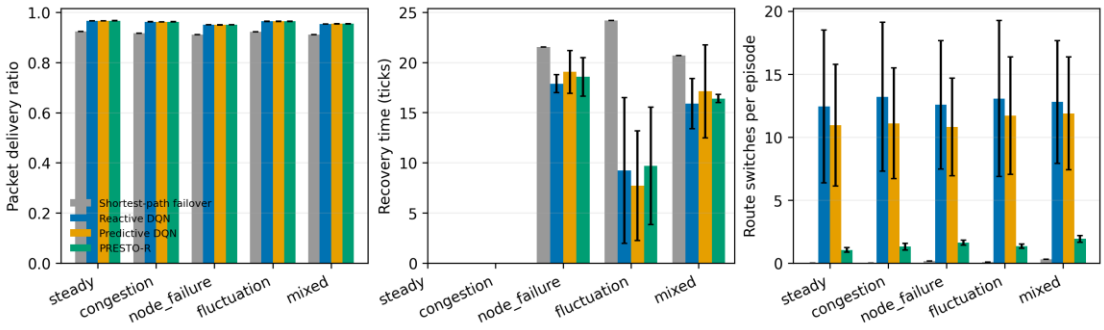


Fig. 7.6 RGG Main Results

Table 7.20 RGG Delivery Delay And Recovery

Method	Scenario	Pdr	Delay Mean	Recovery Time
sp_failover	steady	0.9242 ± NA	68.3566 ± NA	not available
sp_failover	congestion	0.9170 ± NA	70.6413 ± NA	not available
sp_failover	node_failure	0.9116 ± NA	68.4587 ± NA	21.5526 ± NA
sp_failover	fluctuation	0.9230 ± NA	68.1102 ± NA	24.1667 ± NA
sp_failover	mixed	0.9108 ± NA	71.3116 ± NA	20.6959 ± NA
reactive_dqn	steady	0.9665 ± 0.0003	34.6919 ± 0.3298	not available
reactive_dqn	congestion	0.9625 ± 0.0003	35.6699 ± 0.3392	not available
reactive_dqn	node_failure	0.9505 ± 0.0003	36.8101 ± 0.2686	17.8846 ± 0.9031
reactive_dqn	fluctuation	0.9643 ± 0.0004	35.6269 ± 0.4518	9.2333 ± 7.2590

Method	Scenario	Pdr	Delay Mean	Recovery Time
reactive_dqn	mixed	0.9542 ± 0.0002	37.9024 ± 0.3205	15.8897 ± 2.5097
predictive_dqn	steady	0.9664 ± 0.0004	35.2924 ± 0.4202	not available
predictive_dqn	congestion	0.9625 ± 0.0002	36.3546 ± 0.6216	not available
predictive_dqn	node_failure	0.9504 ± 0.0003	37.2256 ± 0.2825	19.0515 ± 2.1342
predictive_dqn	fluctuation	0.9647 ± 0.0005	36.0663 ± 0.3574	7.7180 ± 5.4765
predictive_dqn	mixed	0.9546 ± 0.0007	38.4670 ± 0.3982	17.1224 ± 4.6365
presto_dqn	steady	0.9669 ± 0.0003	34.4730 ± 1.4464	not available
presto_dqn	congestion	0.9627 ± 0.0005	35.5245 ± 1.5505	not available
presto_dqn	node_failure	0.9508 ± 0.0004	36.5316 ± 1.0023	18.5616 ± 1.9305
presto_dqn	fluctuation	0.9647 ± 0.0005	35.5024 ± 1.6147	9.6881 ± 5.8649
presto_dqn	mixed	0.9545 ± 0.0010	38.1368 ± 1.9417	16.3996 ± 0.3937

Table 7.21 RGG Switching Proactivity And Unnecessary Switching

Method	Scenario	Switches	Proactive Rate	Unnecessary Switch Rate
sp_failover	steady	0.0000 ± NA	not available	not available
sp_failover	congestion	0.0000 ± NA	not available	not available
sp_failover	node_failure	0.1550 ± NA	0.0000 ± NA	0.0000 ± NA
sp_failover	fluctuation	0.0650 ± NA	0.0000 ± NA	0.0000 ± NA
sp_failover	mixed	0.2850 ± NA	0.0058 ± NA	0.0000 ± NA
reactive_dqn	steady	12.4383 ± 6.0721	not available	0.6090 ± 0.1239
reactive_dqn	congestion	13.1983 ± 5.9101	not available	0.6268 ± 0.0945
reactive_dqn	node_failure	12.5583 ± 5.1001	0.3853 ± 0.0151	0.6007 ± 0.0860
reactive_dqn	fluctuation	13.0617 ± 6.1823	0.0667 ± 0.0577	0.5861 ± 0.0971

Method	Scenario	Switches	Proactive Rate	Unnecessary Switch Rate
reactive_dqn	mixed	12.7883 ± 4.8685	0.3438 ± 0.0343	0.5656 ± 0.0596
predictive_dqn	steady	10.9417 ± 4.8419	not available	0.6496 ± 0.1253
predictive_dqn	congestion	11.0900 ± 4.4003	not available	0.6616 ± 0.0919
predictive_dqn	node_failure	10.8100 ± 3.8714	0.3817 ± 0.0398	0.6129 ± 0.1033
predictive_dqn	fluctuation	11.7017 ± 4.6742	0.2968 ± 0.0847	0.6378 ± 0.0972
predictive_dqn	mixed	11.8733 ± 4.4776	0.4826 ± 0.1154	0.6037 ± 0.0747
presto_dqn	steady	1.0383 ± 0.1922	not available	0.3146 ± 0.0184
presto_dqn	congestion	1.3033 ± 0.2642	not available	0.3733 ± 0.0195
presto_dqn	node_failure	1.6250 ± 0.1922	0.3607 ± 0.0478	0.2928 ± 0.0337
presto_dqn	fluctuation	1.3267 ± 0.1692	0.0859 ± 0.0834	0.3274 ± 0.0148
presto_dqn	mixed	1.9133 ± 0.2644	0.4159 ± 0.0706	0.2932 ± 0.0256

RGG Scenario Analysis: Steady

RGG steady episodes show PDR 0.9665 for reactive DQN, 0.9664 for predictive DQN, and 0.9669 for PRESTO-R. Their delays are 34.6919, 35.2924, and 34.4730. Switches are 12.4383, 10.9417, and 1.0383. The shortest-path policy has PDR 0.9242 and delay 68.3566.

The result supports lower route churn with comparable delivery in the steady RGG setting. Recovery and proactive rate are unavailable where no qualifying event is recorded. The deterministic heuristic provides a lower-performing route-quality baseline.

RGG Scenario Analysis: Congestion

Under RGG congestion, PDR is 0.9625 for reactive DQN, 0.9625 for predictive DQN, and 0.9627 for PRESTO-R. Delays are 35.6699, 36.3546, and 35.5245. PRESTO-R switches 1.3033 times, compared with 13.1983 for reactive and 11.0900 for predictive. The shortest-path baseline PDR is 0.9170 and delay 70.6413.

The learned policies preserve similar delivery, while PRESTO-R has the lowest switch count. Unnecessary-switch rates are 0.3733 for PRESTO-R, 0.6268 for reactive,

and 0.6616 for predictive. The reported across-seed standard deviations are retained in the full table.

RGG Scenario Analysis: Node Failure

RGG node-failure PDR is 0.9505 for reactive DQN, 0.9504 for predictive DQN, and 0.9508 for PRESTO-R. Delays are 36.8101, 37.2256, and 36.5316. Recovery times are 17.8846, 19.0515, and 18.5616. Switches are 12.5583, 10.8100, and 1.6250.

PRESTO-R retains similar PDR and lowers switching, but recovery is not faster than reactive DQN. Its unnecessary-switch rate is 0.2928, against 0.6007 reactive and 0.6129 predictive. This pattern again supports a stability effect while showing that recovery outcomes depend on scenario and topology.

RGG Scenario Analysis: Fluctuation

RGG fluctuation PDR is 0.9643 for reactive DQN, 0.9647 for predictive DQN, and 0.9647 for PRESTO-R. Delays are 35.6269, 36.0663, and 35.5024. Switch means are 13.0617, 11.7017, and 1.3267. Recovery-time means are 9.2333, 7.7180, and 9.6881.

The route-switch reduction is pronounced, but the recovery summary for PRESTO-R is higher than both comparators. Proactive rate is 0.0859 for PRESTO-R, 0.0667 for reactive DQN, and 0.2968 for predictive DQN. These values argue against describing lower switching as a universal improvement in all dimensions.

RGG Scenario Analysis: Mixed

RGG mixed PDR is 0.9542 for reactive DQN, 0.9546 for predictive DQN, and 0.9545 for PRESTO-R. Delays are 37.9024, 38.4670, and 38.1368. Recovery times are 15.8897, 17.1224, and 16.3996. Switch counts are 12.7883, 11.8733, and 1.9133.

PRESTO-R has a large switching reduction versus predictive DQN and PDR lower by 0.0001. Its unnecessary-switch rate is 0.2932, compared with 0.5656 reactive and 0.6037 predictive. The mixed RGG result aligns with the NSFNET stability pattern, while its recovery time remains longer than reactive DQN.

7.9 Robustness Evaluation

The sudden-failure case removes precursor warning. Reactive DQN PDR is 0.9619, delay 56.6613, switches 18.799, recovery 7.7741, proactive rate 0.4572, and unnecessary-switch rate 0.3444. PRESTO-R PDR is 0.9625, delay 56.3287, switches

5.197, recovery 8.8419, proactive rate 0.3901, and unnecessary-switch rate 0.2344. Thus route stability improves, but recovery is slower and proactive rate lower.

Under twice the measurement noise, reactive DQN records PDR 0.9618, delay 56.5514, switches 19.916, recovery 8.0025, proactive rate 0.4418, and unnecessary-switch rate 0.3656. PRESTO-R records PDR 0.9626, delay 56.3226, switches 5.582, recovery 9.0004, proactive rate 0.3994, and unnecessary-switch rate 0.2762. Mean delivery and delay remain close, with fewer switches under PRESTO-R.

At 150 percent background load, congestion reduces PDR to 0.8728 for reactive DQN and 0.8736 for PRESTO-R. Delay rises to 133.9481 and 130.1015. Switches are 18.846 and 4.985. Recovery times are 16.1781 and 17.0380, while proactive rates are 0.4341 and 0.3972. The stability effect persists under load, but the absolute PDR decrease emphasizes the stress of the modified traffic condition.

The complete robustness CSV contains mean, standard deviation, and training seed count for PDR, delay, switches, unnecessary-switch rate, recovery time, and proactive rate for each method and case. The table below reproduces all saved rows rather than only a selected subset.

Table 7.22 Evaluation-Only Robustness Results

Case	Method	Metric	Mean	SD	Seeds
sudden	reactive_sudden	pdr	0.961937	0.000774456	5
sudden	reactive_sudden	delay_mean	56.6613	0.713338	5
sudden	reactive_sudden	switches	18.799	1.45753	5
sudden	reactive_sudden	unnecessary_switch_rate	0.344362	0.0278885	5
sudden	reactive_sudden	recovery_time	7.77407	0.325827	5
sudden	reactive_sudden	proactive_rate	0.457156	0.03387	5
sudden	presto_sudden	pdr	0.962476	0.000523774	5
sudden	presto_sudden	delay_mean	56.3287	0.961025	5

Case	Method	Metric	Mean	SD	Seeds
sudden	presto_sudden	switches	5.197	1.62755	5
sudden	presto_sudden	unnecessary_switch_rate	0.234397	0.0493655	5
sudden	presto_sudden	recovery_time	8.8419	0.680961	5
sudden	presto_sudden	proactive_rate	0.390095	0.0524359	5
noise2x	reactive_noise2x	pdr	0.961808	0.000735908	5
noise2x	reactive_noise2x	delay_mean	56.5514	0.554837	5
noise2x	reactive_noise2x	switches	19.916	1.31942	5
noise2x	reactive_noise2x	unnecessary_switch_rate	0.365635	0.0315399	5
noise2x	reactive_noise2x	recovery_time	8.00251	0.470984	5
noise2x	reactive_noise2x	proactive_rate	0.441779	0.0493953	5
noise2x	presto_noise2x	pdr	0.962598	0.000524742	5
noise2x	presto_noise2x	delay_mean	56.3226	0.729433	5
noise2x	presto_noise2x	switches	5.582	1.69677	5
noise2x	presto_noise2x	unnecessary_switch_rate	0.27621	0.0292546	5
noise2x	presto_noise2x	recovery_time	9.00036	0.842798	5
noise2x	presto_noise2x	proactive_rate	0.399435	0.0598887	5
load150	reactive_load150	pdr	0.872821	0.00104575	5
load150	reactive_load150	delay_mean	133.948	0.749523	5
load150	reactive_load150	switches	18.846	2.17085	5
load150	reactive_load150	unnecessary_switch_rate	0.121939	0.0156974	5

Case	Method	Metric	Mean	SD	Seeds
load150	reactive_load150	recovery_time	16.1781	0.263317	5
load150	reactive_load150	proactive_rate	0.434096	0.0262495	5
load150	presto_load150	pdr	0.873628	0.00133692	5
load150	presto_load150	delay_mean	130.102	1.33738	5
load150	presto_load150	switches	4.985	1.8631	5
load150	presto_load150	unnecessary_switch_rate	0.0848134	0.0280157	5
load150	presto_load150	recovery_time	17.038	1.92648	5
load150	presto_load150	proactive_rate	0.397199	0.0387805	5

7.10 Success Criteria

The predeclared criteria distinguish performance against reactive and predictive DQN, steady-state delay, predictor onset F1, and reproducibility. PRESTO-R passes the specified predictive comparisons for mixed and fluctuation scenarios on switch reduction with comparable PDR. It passes the steady delay limit and reproducibility criterion. It fails the comparison against reactive DQN in node failure and fluctuation and fails the onset-F1 threshold.

The failed node-failure criterion records PDR 0.9667 versus 0.9662 and recovery 9.440 versus 8.915, with Holm p-values of 1 for PDR and 0.2617 for recovery. The fluctuation criterion records PDR 0.9749 versus 0.9740 and recovery 7.844 versus 7.386, with Holm p-values 0.01042 for PDR and 1 for recovery. The predictor criterion records GRU validation onset F1 0.5296 against 0.7000. These are reported as failures even though some point estimates favor PRESTO-R.

The source success sheet and the T3 summary table contain slightly different recovery aggregates. T3 reports PRESTO-R and reactive means of 9.4236 and 8.8951 for node failure, and 7.9852 and 7.4906 for fluctuation. The success sheet reports 9.440 and 8.915, and 7.844 and 7.386. These values are preserved in their source contexts. They

reflect different summary outputs and should be reconciled against raw episode records before any submission that requires one single recovery estimate.

Table 7.23 Predeclared Success Criteria

Check	Result	Details
PRESTO-R vs reactive in node_failure	FAIL	PDR 0.9667 vs 0.9662; recovery 9.440 vs 8.915; Holm p PDR=1, recovery=0.2617
PRESTO-R vs reactive in fluctuation	FAIL	PDR 0.9749 vs 0.9740; recovery 7.844 vs 7.386; Holm p PDR=0.01042, recovery=1
PRESTO-R vs predictive in mixed	PASS	Switches 5.197 vs 20.988; PDR 0.9625 vs 0.9608
PRESTO-R vs predictive in fluctuation	PASS	Switches 5.557 vs 23.419; PDR 0.9749 vs 0.9726
Steady-state delay	PASS	PRESTO-R delay 46.2766; limit 50.9369 (reactive x 1.10)
Predictor onset F1	FAIL	GRU validation onset F1=0.5296; threshold=0.7000
Reproducibility	PASS	Step and event parquet hashes match on a fresh rerun

Table 7.24 Success-Criteria And Summary Aggregation Difference

Scenario	Source	PRESTO-R Recovery	Reactive Recovery	Reporting Note
Node failure	Success criteria	9.440	8.915	Paired success summary values
Node failure	T3 mean over seed summaries	9.4236	8.8951	Across-training-seed summary table
Fluctuation	Success criteria	7.844	7.386	Paired success summary values
Fluctuation	T3 mean over seed summaries	7.9852	7.4906	Across-training-seed summary table

7.11 Findings by Scenario

Across steady conditions, the switch penalty reduces route changes while preserving or slightly improving mean delivery and delay. There are no qualifying event-based recovery measures in steady runs. Under congestion, PRESTO-R retains similar PDR, lowers delay slightly, and makes substantially fewer switches. These findings are consistent on both topologies, though absolute delays and switch baselines differ.

Under node failure, route-switch counts remain much lower, but recovery is not consistently improved. The project's predeclared comparison against reactive DQN fails. In fluctuation, PRESTO-R shows a strong switch reduction and close PDR, but the success criterion against reactive DQN still fails under its paired rule. This is an example of why a summary mean alone cannot override a planned inferential criterion.

Mixed scenarios combine the strongest evidence for stability with useful limitations. On NSFNET, switches fall from 20.988 to 5.197 relative to predictive DQN while PDR rises by 0.0017. On RGG, switches fall from 11.873 to 1.913 while PDR changes by -0.0001. Recovery is higher for PRESTO-R than reactive DQN in the mixed summaries, so route stability is not equivalent to faster recovery.

Robustness tests show that reduced switching persists under sudden failures, doubled noise, and increased background load. They also show lower proactive rates or slower recovery in some conditions, and a large PDR decline under the 150 percent load condition for both policies. These checks strengthen the stability observation but do not erase the predictor target failure or prove external generalization.

7.12 Threats to Validity

Construct validity is limited by the metric definitions and synthetic event process. Unnecessary switching depends on an H-tick lookahead, proactive behavior depends on failure truth, and recovery is based on a loss threshold of 0.02. Different operational definitions could change metric values. Predictor labels depend on simulator thresholds for bad loss and bad delay.

Internal validity depends on matched seeds, resolved configurations, and consistent model checkpoints. The project records separated training, validation, and test seed ranges and reports a fresh-rerun hash match. However, table-level recovery

summaries differ across success criteria and T3. This discrepancy is explicitly retained and should be reconciled from the raw metric files if a single value is required.

External validity is limited to one fluid simulator, one managed flow, synthetic failures, two topologies, and the selected seed ranges. The results do not establish production performance, protocol convergence, or transfer to networks with different traffic and failures. NS-3 validation is optional and absent.

Statistical validity is limited by the number of training seeds, subgroup sizes, available event episodes, multiple comparisons, and incomplete factorial ablation. The Holm adjustment reduces family-wise error across the specified comparison family, but it cannot correct for every design omission. The missing no-prediction plus penalty cell prevents a full causal decomposition.

CHAPTER 8
SUMMARY

PRESTO-R implements a predictive routing design in which GRU-based link risk augments path observations and DQN selects a route under a switching penalty. The evaluation shows that the stability objective was achieved in the tested simulator: PRESTO-R substantially reduces route switching relative to predictive DQN on mixed episodes for both NSFNET and RGG. Delivery remains close to learned baselines. The evidence is not uniformly positive. The GRU onset-F1 target is missed, comparisons against reactive DQN fail in two predeclared conditions, and the complete prediction-by-penalty factorial is missing.

8.1 Objective Achievement

The simulator objective is achieved through a seeded discrete-time fluid model, two topologies, five scenarios, hidden link health, noisy measurements, event schedules, and a shared routing environment. The predictor objective is partially achieved: GRU and logistic-regression models are trained, evaluated, and integrated as risk providers, but the primary onset-F1 target is not reached.

The routing objective is achieved with reactive, predictive, and stability-aware DQN policies plus a shortest-path failover baseline. The main mixed-scenario stability comparison shows fewer route changes for PRESTO-R and similar PDR. The full result tables, paired tests, sensitivity analyses, and robustness results are saved and reported. The missing factorial ablation limits causal attribution of the combined design.

The reproducibility and reporting objective is supported by resolved configurations, separate seed ranges, a resumable evaluation grid, a 38,400-row episode-metric table, figures, checksums, and a recorded hash match for fresh step and event outputs. The report preserves failures and uncertainty instead of presenting only favorable evidence.

8.2 Conclusions

In the tested simulator, adding the selected switching cost to the risk-aware DQN substantially reduces route churn. On NSFNET mixed episodes, PRESTO-R averages 5.197 switches compared with 20.988 for predictive DQN, with PDR 0.9625 compared

with 0.9608. On RGG mixed episodes, PRESTO-R averages 1.913 switches compared with 11.873, with PDR 0.9545 compared with 0.9546.

This conclusion is deliberately bounded. PRESTO-R is not uniformly superior to reactive DQN on every predeclared criterion. The onset predictor fails its F1 target. Some recovery outcomes are not improved, and the full causal ablation is unavailable. The correct conclusion is a demonstrated route-stability benefit under specific simulated conditions, coupled with unresolved prediction and recovery limitations.

8.3 Recommendations

The next experiment should complete the two-by-two comparison of prediction presence and switch-penalty presence, keeping training budget and other hyperparameters matched. It should preregister the policy-comparison criteria, specify which aggregation supports each criterion, and report all event-bearing recovery sample counts. This would clarify whether the combined benefit comes from predictor features, switching cost, or their interaction.

The predictor should be improved and re-evaluated on onset samples, with calibration and subgroup metrics retained. Sudden events should remain a separate case because no precursor can be learned when there is no warning signal. Scenario and topology splits should remain isolated to test transfer rather than only within-distribution performance.

A packet-level or real-trace evaluation should follow after the simulator findings are stable. A multi-flow setting should measure fairness and interference. Any operational recommendation should require instrumentation, safety fallback, route-change validation, and an offline shadow evaluation before live control.

8.4 Future Work

Planned extensions include multiple managed flows, multipath splitting, packet-level or NS-3 validation, online predictor updates, domain randomization, broader topology sets, and more robust uncertainty estimation. The optional NS-3 phase is not completed in the current project. It should be reported separately after a simulator-to-packet comparison is available.

Future reporting should reconcile the recovery values between the success criteria summary and T3 summaries, preferably from a single scripted aggregation pipeline. It

should also archive the exact report-generation version, summary CSV hashes, and predictor metric files with the final document. These steps would make all table values traceable to a unique saved computation.

CHAPTER 9
REFERENCES

The references below support the algorithmic context and identify the project artifacts from which the system details and measured results were taken. Academic references are not presented as evidence that the project reproduces their experiments. Repository outputs are cited as the source of the numerical results.

[1] V. Mnih et al., “Human-level control through deep reinforcement learning,” Nature, vol. 518, pp. 529-533, 2015. doi:10.1038/nature14236.

[2] K. Cho, B. van Merrienboer, C. Gulcehre, D. Bahdanau, F. Bougares, H. Schwenk, and Y. Bengio, “Learning Phrase Representations using RNN Encoder-Decoder for Statistical Machine Translation,” Proceedings of EMNLP, 2014. doi:10.48550/arXiv.1406.1078.

[3] J. Y. Yen, “Finding the K Shortest Loopless Paths in a Network,” Management Science, vol. 17, no. 11, pp. 712-716, 1971. doi:10.1287/mnsc.17.11.712.

[4] PRESTO-R source repository, simulator, predictor, agent, policy, evaluation, and test modules, Computer Networks project workspace, accessed October 2026.

[5] PRESTO-R configuration files under configs/, including base, topology, scenario, predictor, agent, and evaluation-grid settings, Computer Networks project workspace, accessed October 2026.

[6] PRESTO-R saved evaluation tables under results/tables/ and results/predictor/, including topology summaries, paired tests, robustness, horizon, lambda, and predictor metrics, Computer Networks project workspace, accessed October 2026.

[7] PRESTO-R generated report and success criteria, results/report.md and results/success_criteria.md, Computer Networks project workspace, accessed October 2026.

[8] NumPy 1.26.4; pandas 2.2.2; SciPy 1.13.1; NetworkX 3.3; Gymnasium 1.0.0; Stable-Baselines3 2.4.0; PyTorch 2.4.1; scikit-learn 1.5.1; Matplotlib 3.9.2; and PyArrow 17.0.0, versions recorded in requirements.txt.

[9] Vellore Institute of Technology, School of Information Technology and Engineering, BCSE308P Computer Networks Course Project Evaluation Sheet, Fall Semester 2026-2027, supplied course material.

[10] VIT course project report sample, “BCSE308P Course Project,” supplied in review docs, used for report structure, typography, margins, page numbering, and caption rules.

APPENDIX A REPRODUCTION AND DATA DICTIONARY

This appendix records the primary project commands and a compact dictionary of the principal result fields. It supplements the report and does not replace the saved repository outputs. Commands are included as reproducibility evidence; executing them requires the pinned dependencies, model artifacts, and raw result files described in the project README.

A.1 Reproduction Commands

The repository supports a quick environment check, predictor training, routing-agent training, full grid evaluation, metrics aggregation, statistical analysis, success checking, figure generation, report export, and model packaging. The sequence below follows the saved README. Full-grid commands depend on trained policies and raw logs already being present.

Command

```
python -m pytest
```

Command

```
python -m scripts.smoke_env
```

Command

```
python -m presto.predictor.dataset --config configs/predictor/gru.yaml
```

Command

```
python -m presto.predictor.train --config configs/predictor/gru.yaml
```

Command

```
python -m presto.predictor.train --config configs/predictor/logreg.yaml
```

Command

```
python -m presto.agents.train --config configs/agent/reactive_dqn.yaml --seed 0
```

Command

```
python -m presto.agents.train --config configs/agent/predictive_dqn.yaml --seed 0
```

Command

```
python -m presto.agents.train --config configs/agent/presto_dqn.yaml --seed 0
```

Command

```
python -m presto.eval.run_grid --grid configs/grid/main.yaml --workers 4
```

Command

```
python -m presto.eval.metrics --raw-dir results/raw --out
results/episode_metrics.parquet
```

Command

python -m presto.eval.stats --metrics results/episode_metrics.parquet --out-dir results/tables

Command

python -m presto.eval.check_success

Command

python -m presto.eval.plots

Command

python -m presto.eval.report

Command

python -m scripts.export_report_pdf

Command

python -m scripts.package_models

A.2 Data Dictionary

The saved episode_metrics.parquet contains one row per evaluated episode and identifies method, topology, scenario, training seed, scenario seed, horizon, and lambda. The following definitions describe the main aggregate metrics used in the report.

Table 0.1 Metric Data Dictionary

Field	Definition
pdr	Total delivered traffic divided by total offered demand.
delay_mean	Delivered-traffic-weighted mean path delay.
throughput	Mean delivered traffic rate.
switches	Number of route changes within an episode.
unnecessary_switch_rate	Fraction of switches for which the previous route remains healthy through horizon H.
recovery_time	Mean ticks from qualifying failure to first valid selected route at or below the QoS loss threshold.
proactive_rate	Share of qualifying failures for which the route had already left the failing link.
energy	Sum of selected-path hop counts over the episode.
reward	Sum of per-step reward.
failure_events	Count of qualifying true link up-to-down transitions.
scheduled_events	Number of scheduled simulator events associated with the episode.

A.3 Project Artifact Map

configs/ contains resolved experiment settings; presto/ contains simulator, predictor, policy, agent, and evaluation modules; tests/ contains automated tests; runs/ stores training checkpoints and logs; data/ stores predictor datasets; results/ contains raw evaluation, episode metrics, tables, figures, and report artifacts; models/ contains the curated release bundle and checksum manifest. The model archive is local-only because no remote release destination is configured.